CHAPTER 13

Sights and Sounds

KEY TERMS

image processing
sound processing
pixels
true color
RGB
colormap
Graphical User Interfaces
event
callback function
event-driven programming
components
sound wave
sampling frequency

CONTENTS

13.1 Image Processing 443
13.2 Introduction to GUIs 456
13.3 GUIDE and App Designer ... 475
13.4 Sound Files 488
Summary 491
Common Pitfalls 491
Programming Style Guidelines 491

The MATLAB® product has functions that manipulate audio or sound files, and also images. MATLAB also has the capability to produce sophisticated graphical user interfaces, or GUIs. GUIs allow users to graphically interact with programs using graphical objects such as push buttons and sliders.

This chapter will start with the introduction of ***image processing*** functions, and the two basic methods for representing color in images.

Next, we will introduce the topic of GUIs from a programming standpoint. Since the focus of this book is programming constructs, the programming aspects of creating GUIs are the main part of this section. However, as of R2014b, the code produced by the graphical user interface development environment, or GUIDE, is easier to understand and modify than previous versions. Also, the exciting new App Designer was introduced in R2016a. This is similar to GUIDE but has much greater functionality and ease of use. Both GUIDE and the App Designer will be introduced.

The chapter ends with a brief introduction to some of the ***sound processing*** functions.

13.1 IMAGE PROCESSING

Color images are represented as grids, or matrices, of picture elements called ***pixels***. In MATLAB, an image is represented by a matrix in which each element

MATLAB®. http://dx.doi.org/10.1016/B978-0-12-804525-1.00013-1
© 2017 Elsevier Inc. All rights reserved.

corresponds to a pixel in the image. Each element that represents a particular pixel stores the color for that pixel. There are two basic ways in which the color can be represented:

- ***true color*** or ***RGB***, in which the three color components are stored (red, green, and blue, in that order) in layers in a three-dimensional matrix
- index into a ***colormap***, in which the value stored for each pixel is an integer that refers to a row in another matrix called a colormap; the colormap stores the red, green, and blue components in three separate columns

Thus, for an image that has $m \times n$ pixels, there are two methods for representing color.

In the true color or RGB method, all of the information is stored in one 3D matrix with a size $m \times n \times 3$. The first two indices represent the coordinates of the pixel. The third index is the color component, so `(:,:,1)` is the red, `(:,:,2)` is the green, and `(:,:,3)` is the blue component.

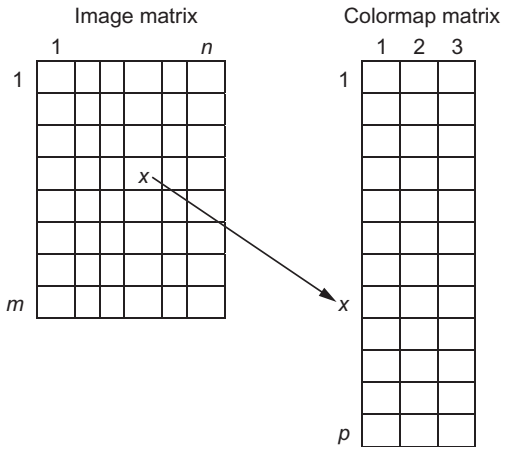
The colormap method instead uses two separate matrices: the image matrix is a 2D matrix with a size of $m \times n$. Every element in this matrix of integers is an index into a colormap matrix, which is of the size $p \times 3$ (where p is the number of colors available in that particular colormap). Each row in the colormap has three numbers representing one color: first the red, then the green, and then the blue component.

Typically colormaps use **double** values in the range from 0 to 1, and RGB matrices use the type **uint8**. These conventions are used in the next two sections that introduce these methods. In Section 13.1.4, alternate number types are explained.

13.1.1 Colormaps

When an image is represented using a colormap, there are two matrices:

- The colormap matrix, which has dimensions $p \times 3$, where p is the number of available colors; every row stores a separate color by storing three real numbers in the range from 0 to 1, representing the red, green, and blue components of the color.
- The image matrix, with dimensions $m \times n$; every element is an index into a particular row in the colormap, which means that it is an integer in the range 1 to p.



MATLAB has several built-in colormaps that are named; these can be seen and can be set using the built-in function **colormap**. The reference page on **colormap** displays them. Calling the function **colormap** without passing any arguments will return the current colormap. The default colormap is a map named **parula**, which has 64 colors. In versions of MATLAB before R2014b the default was the colormap named **jet**, which is still an available colormap.

The following stores the current colormap in a variable *map*, gets the size of the matrix (which will be the number of rows in this matrix or, in other words, the number of colors, by three columns), and displays the first five rows in this colormap. If the current colormap is the default **parula**, the following will be the result:

```
>> map = colormap;
>> [r, c] = size(map)
r =
    64
c =
     3
>> map(60:64, :)
ans =
    0.9626    0.8705    0.1309
    0.9589    0.8949    0.1132
    0.9598    0.9218    0.0948
    0.9661    0.9514    0.0755
    0.9763    0.9831    0.0538
```

This shows that there are 64 rows or, in other words, 64 colors, in this particular colormap. It also shows that the last five colors are shades of yellow (the combination of almost full red and green but very little blue). Note that **parula** is

actually a function that returns a colormap matrix. Passing no arguments results in the 64×3 matrix shown here, although the number of desired colors can be passed as an argument to the **parula** function.

The format of calling the **image** function is

```
image(mat)
```

where the matrix *mat* represents the colors in an $m \times n$ image ($m \times n$ pixels in the image). If the matrix has the size $m \times n$, then each element is an index into the current colormap.

One way to display the colors in the default **parula** colormap (which has 64 colors) is to create a matrix that stores the values 1 through 64, and pass that to the **image** function; the result is shown in Fig. 13.1. When the matrix is passed to the **image** function, the value in each element in the matrix is used as an index into the colormap.

For example, the value in imagemat(1,2) is 9 so the color displayed in location (1,2) in the image will be the color represented by the ninth row in the colormap. By using the numbers 1 through 64, we can see all colors in this colormap.

```
>> imagemat = reshape(1:64, 8,8)
imagemat =

     1     9    17    25    33    41    49    57
     2    10    18    26    34    42    50    58
     3    11    19    27    35    43    51    59
     4    12    20    28    36    44    52    60
     5    13    21    29    37    45    53    61
     6    14    22    30    38    46    54    62
     7    15    23    31    39    47    55    63
     8    16    24    32    40    48    56    64
>> image(imagemat)
```

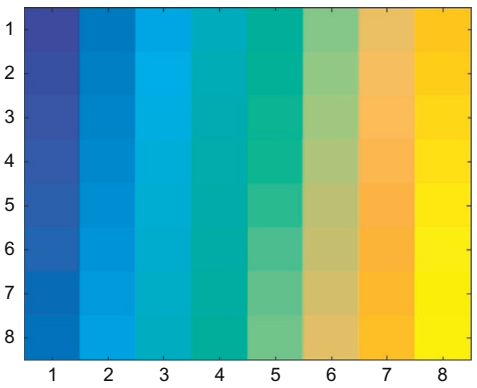


FIGURE 13.1
Columnwise display of the 64 colors in the **parula** colormap.

As with plots, the result of calling the **image** function can be stored in a variable; this allows the properties of the image to be inspected and/or modified using either the dot notation or **get** and **set**.

```
>> im = image(imagemat);
```

In order to see the colors in the colormap **jet** instead, we could make **jet** the current colormap and re-display the image matrix, as seen in Fig. 13.2.

```
>> colormap(jet)
>> image(imagemat)
```

Another example creates a 5×5 matrix of random integers in the range from 1 to the number of colors (stored in a variable r); the resulting image appears in Fig. 13.3 (assuming that the colormap is still **jet**).

```
>> mat = randi(r,5)
    54    33    13    45    32
     2    46    44    25    58
    44    28    20    56    53
    25    20    35    55    42
    54    13    10    38    53
>> image(mat)
```

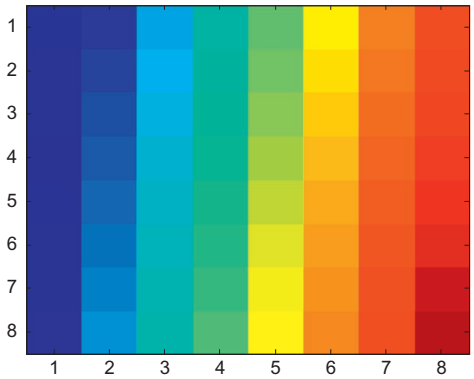


FIGURE 13.2
Columnwise display of the 64 colors in the **jet** colormap.

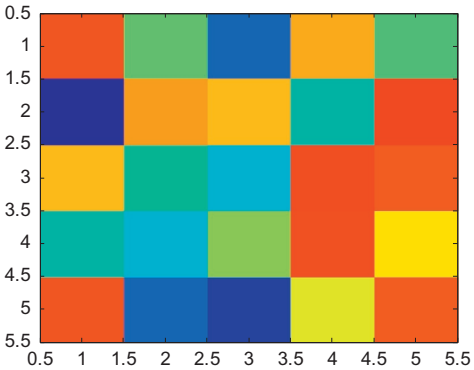


FIGURE 13.3
A 5×5 display of random colors from the **jet** colormap.

Of course, these "images" are rather crude; the elements representing the pixel colors are quite large blocks. A larger matrix would result in something more closely resembling an image, as shown in Fig. 13.4.

```
>> mat = randi(r,500);
>> image(mat)
```

Although MATLAB has built-in colormaps, it is also possible to create others using any color combinations. For example, the following creates a customized

colormap with just three colors: black, white, and red. This is then set to be the current colormap by passing the colormap matrix to the **colormap** function. Then, a 40×40 matrix of random integers, each in the range from 1 to 3 (as there are just three colors), is created, and that is passed to the **image** function; the results are shown in Fig. 13.5.

```
>> mycolormap = [0 0 0; 1 1 1; 1 0 0]
mycolormap =
     0     0     0
     1     1     1
     1     0     0
>> colormap(mycolormap)
>> mat = randi(3,40);
>> image(mat)
```

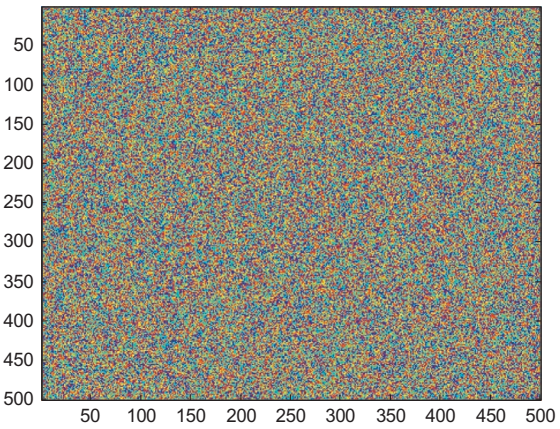


FIGURE 13.4
A *500* × *500* display of random colors.

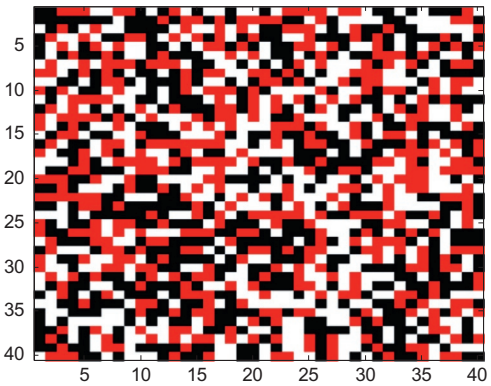


FIGURE 13.5
Random colors from a custom colormap.

The numbers in the colormap do not have to be integers; real numbers represent different shades as seen with the default colormap **parula**. For example, the following colormap gives us a way to visualize different shades of red as shown in Fig. 13.6.

```
>> colors = [0 0 0; 0.2 0 0; 0.4 0 0; ...
             0.6 0 0; 0.8 0 0; 1 0 0];
>> colormap(colors)
>> vec = 1:length(colors);
>> image(vec)
```

PRACTICE 13.1

Given the following colormap, "draw" the scene shown in Fig. 13.7. (Hint: Preallocate the image matrix. The fact that the first color in the colormap is white, makes this easier.)

```
>> mycolors = [1 1 1; 0 1 0; 0 0.5 0; ...
               0 0 1; 0 0 0.5; 0.3 0 0];
```

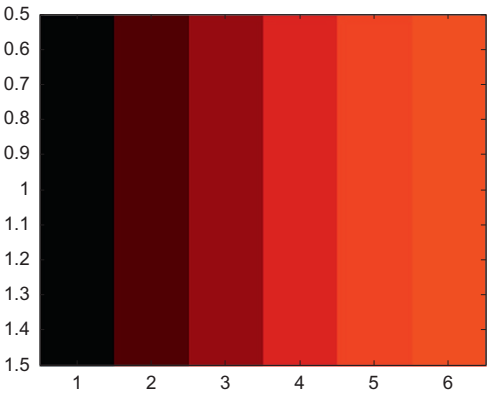


FIGURE 13.6
Shades of red.

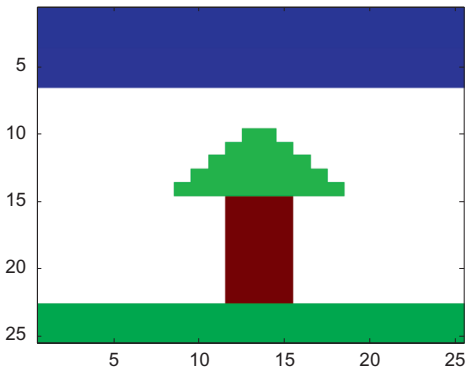


FIGURE 13.7
Draw this tree with grass and sky.

Colormaps are used with many plot functions. Generally, the plots shown assume the default colormap **parula**, but the colormap can be modified. For example, plotting a 3D object using **surf** or **sphere**, and displaying a **colorbar** would normally display the parula colors. The following is an example of modifying this to use the colormap **pink**, as shown in Fig. 13.8.

```
>> [x,y,z] = sphere(20);
>> colormap(pink)
>> surf(x,y,z)
>> title('Pink sphere')
>> colorbar
```

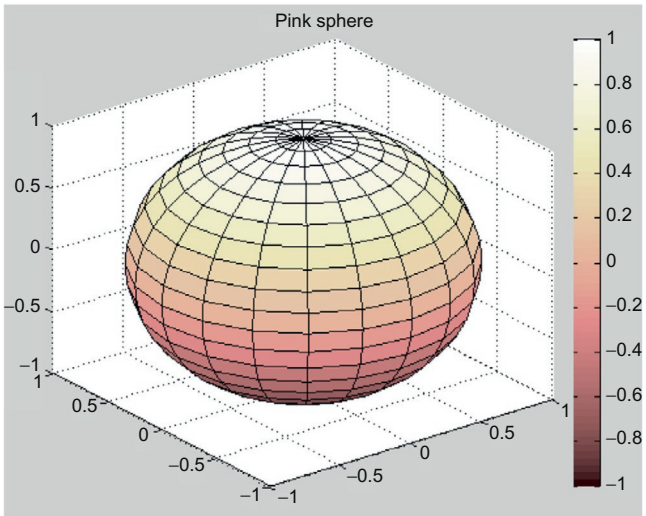


FIGURE 13.8
Pink colormap for sphere function.

As of R2014b, multiple colormaps can be displayed in a single Figure Window by passing the axes handle to the **colormap** function as shown in Fig. 13.9.

```
[x,y,z] = sphere(20);
ax1 = subplot(1,2,1)
colormap(ax1,pink)
surf(x,y,z);
title('Pink sphere')
ax2 = subplot(1,2,2)
colormap(ax2,jet)
surf(x,y,z);
title('Jet sphere')
```

13.1.2 True Color Matrices

True color matrices, or ***RGB matrices***, are another way to represent images. True color matrices are 3D matrices. The first two coordinates are the coordinates of the pixel. The third index is the color component; `(:,:,1)` is the red, `(:,:,2)` is the green, and `(:,:,3)` is the blue component.

In an 8-bit RGB image, each element in the matrix is of the type **uint8**, which is an unsigned integer type storing values in the range from 0 to 255. The minimum value, 0, represents the darkest hue available, so all 0s results in a black pixel. The maximum value 255, represents the brightest hue. For example, if the values for a given pixel coordinates *px* and *py* are: (px,py,1) is 255, (px,py,2) is 0,

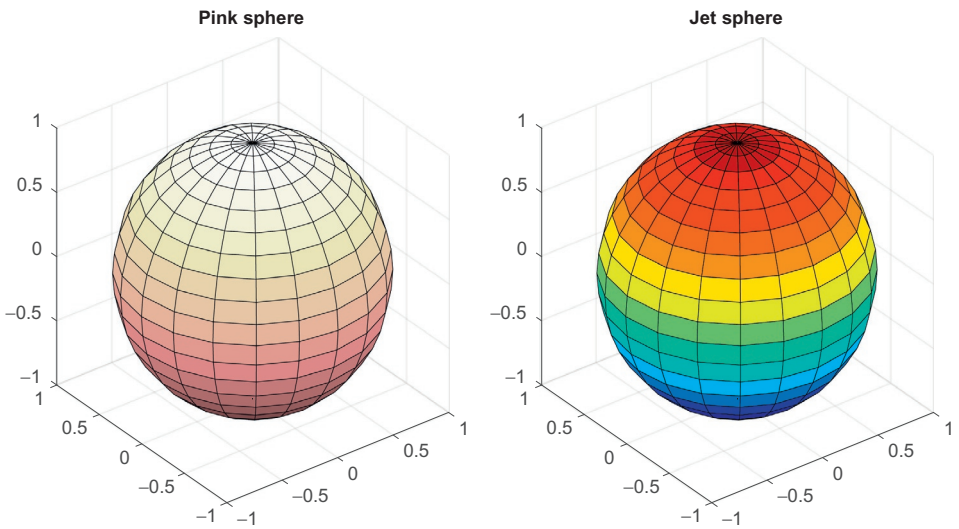


FIGURE 13.9
Subplot with multiple colormaps.

and (px,py,3) is 0, then that pixel will be bright red. All 255s results in a white pixel.

The **image** function displays the information in the 3D matrix as an image. (Note: if a 2D image matrix is passed to **image**, it uses the current colormap to display the image, but if a 3D image matrix is passed to **image**, it ignores the colormap and instead uses the information from the 3D matrix to determine the color of each pixel.)

For example, the following creates a 2×2 image as shown in Fig. 13.10. The matrix is $2 \times 2 \times 3$ where the third dimension is the color. The pixel in location (1,1) is red, the pixel in location (1,2) is blue, the pixel in location (2,1) is green, and the pixel in location (2,2) is black. To make the pixel in location (1,1) red, we could execute the following three statements, which would make the red layer the brightest possible, with no green or blue:

```
>> mat(1,1,1) = 255;
>> mat(1,1,2) = 0;
>> mat(1,1,3) = 0;
```

Similarly we could execute three statements to set the color for each of the other pixels. However, it is much simpler to start by preallocating the entire image matrix to all zeros, and then changing individual values to 255 as necessary. In versions prior to R2016a, it is necessary to cast the matrix to the type **uint8** before passing it to the **image** function.

```
>> mat = zeros(2,2,3);
>> mat(1,1,1) = 255;
>> mat(1,2,3) = 255;
>> mat(2,1,2) = 255;
>> mat = uint8(mat);
>> image(mat)
```

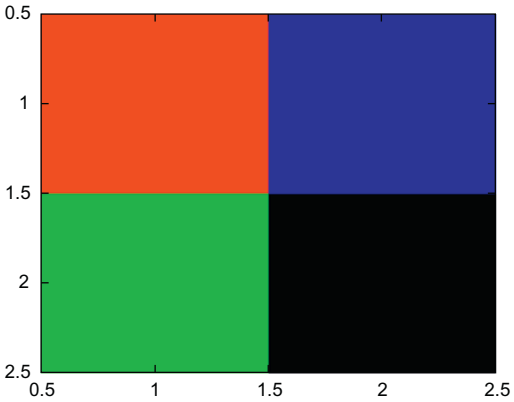


FIGURE 13.10
Image from a true color matrix.

The following shows how to separate the red, green, and blue components from an image matrix. In this case we are using the "image" matrix *mat*, and then use **subplot** to display the original matrix and the red, green, and blue component matrices, as shown in Fig. 13.11.

```
matred = uint8(zeros(2,2,3));
matred(:,:,1) = mat(:,:,1);
matgreen = uint8(zeros(2,2,3));
matgreen(:,:,2) = mat(:,:,2);
matblue = uint8(zeros(2,2,3));
matblue(:,:,3) = mat(:,:,3);
subplot(2,2,1)
image(mat)
subplot(2,2,2)
image(matred)
subplot(2,2,3)
image(matgreen)
subplot(2,2,4)
image(matblue)
```

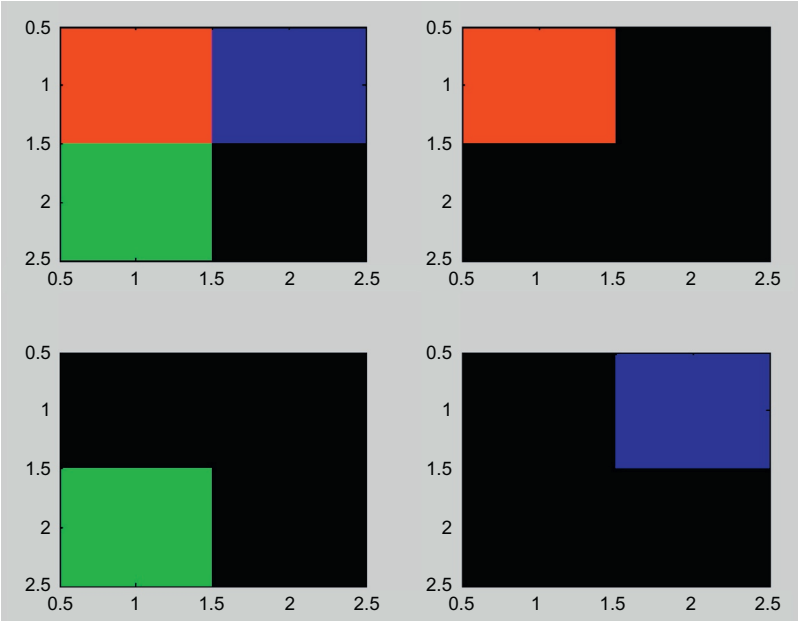


FIGURE 13.11
Separating red, green, and blue components.

Superimposing the images from the three matrices *matred*, *matgreen*, and *matblue* would be achieved by simply adding the three arrays together. The following would result in the image from Fig. 13.10:

```
>> image(matred+matgreen+matblue)
```

PRACTICE 13.2

Create the 3×3 (×3) true color matrix shown in Fig. 13.12 (the axes are defaults). Use the type **uint8**.

13.1.3 Image Files

Images that are stored in various formats, such as JPEG, TIFF, PNG, GIF, and BMP, can be manipulated in MATLAB. Built-in functions, such as **imread** and **imwrite**, read from and write to various image file formats. Some images are stored as unsigned 8-bit data (**uint8**), some as unsigned 16-bit (**uint16**), and some are stored as **double**.

For example, the following reads a JPEG image into a 3D matrix; it can be seen from the **size** and **class** functions that this was stored as a **uint8** RGB matrix.

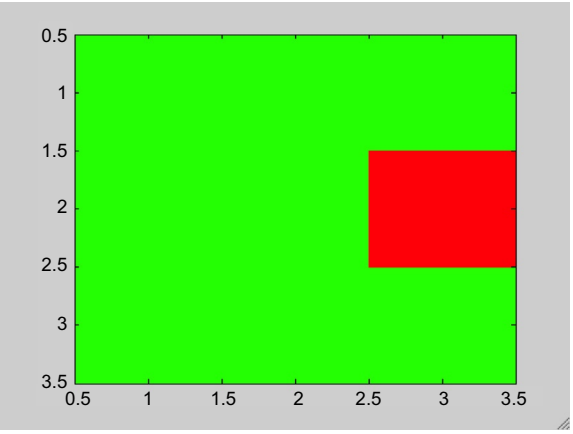


FIGURE 13.12
Create this true color matrix.

```
>> porchimage = imread('snowyporch.JPG');
>> size(porchimage)
ans =
        2848        4272           3
>> class(porchimage)
ans =
uint8
```

The image is stored as a true color matrix and has 2848×4272 pixels. The image function displays the matrix as an image, as shown in Fig. 13.13.

```
>> image(porchimage)
```

The function **imshow** can also be used to display an image. Some image functions, such as **imshow**, **rgb2gray** (converts from RGB to a gray scale), and **im2double** (converts an image matrix to **double**) are now in MATLAB; prior to R2014b they were only in the Image Processing Toolbox.

The image can be manipulated by modifying the numbers in the image matrix. For example, multiplying every number by 0.5 will result in a range of values from 0 to 128 instead of from 0 to 255. As the larger numbers represent brighter hues, this will have the effect of dimming the hues in the pixels, as shown in Fig. 13.14.

```
>> dimmer = 0.5*porchimage;
>> image(dimmer)
```

The function **imwrite** is used to write an image matrix to a file in a specified format:

```
>> imwrite(dimmer, 'dimporch.JPG')
```



FIGURE 13.13
Image from a JPEG file displayed using **image**.



FIGURE 13.14
Image dimmed by manipulating the matrix values.

Images can also be stored as an indexed image rather than RGB. In that case, the colormap is usually stored with the image and will be read in by the **imread** function.

13.1.4 Alternate Number Types for Image Matrices

In the previous two sections, we have described the colormap method using **double** values in the range from 0 to 1 in the colormap matrix and the RGB method using the type **uint8** in the 3D matrix. However, it is not necessary to use these particular types.

Usually, the image matrices used in conjunction with a colormap are the type **double**, as described in Section 13.1.2, and the integers stored in the matrices range from 1 to p, where p is the number of colors in the current colormap. However, it is possible for the image matrix to store the data as the type **uint8** or **uint16**. In that case, the integers stored would range from 0 to $p-1$, and the **image** function adjusts appropriately (a 0 maps to the first color, 1 maps to the second color, and so forth).

The numbers in an RGB matrix can be of the types **uint8**, **uint16**, or **double**.

The image seen in Fig. 13.10 could also be created as a 16-bit image in which the range of values would be from 0 to 65,535 instead of from 0 to 255.

```
>> clear
>> mat = zeros(2,2,3);
>> mat(1,1,1) = 65535;
>> mat(1,2,3) = 65535;
>> mat(2,1,2) = 65535;
>> mat = uint16(mat);
>> image(mat)
```

In an RGB image matrix in which the numbers range from 0 to 1, the default type is **double** so it is not necessary to typecast the matrix variable. The following would also create the image seen in Fig. 13.9.

```
>> clear
>> mat = zeros(2,2,3);
>> mat(1,1,1) = 1;
>> mat(1,2,3) = 1;
>> mat(2,1,2) = 1;
>> image(mat)
```

The **image** function determines the type of the image matrix and adjusts the colors accordingly when displaying the image.

13.2 INTRODUCTION TO GUIs

GUIs are essentially objects that allow users to have input using graphical interfaces, such as pushbuttons, sliders, radio buttons, toggle buttons, pop-up menus, and so forth. GUIs are an implementation of the handle classes that were introduced in Chapter 11, so there is a hierarchy. For example, the parent may be a Figure Window and its children would be graphics objects, such as pushbuttons and text boxes.

The parent user interface object can be a **figure**, **uipanel**, or **uibuttongroup**. A **figure** is a Figure Window created by the **figure** function. A **uipanel** is a means of grouping together user interface objects (the "ui" stands for user interface). A **uibuttongroup** is a means of grouping together buttons (both radio buttons and toggle buttons).

In MATLAB there are two basic methods for creating GUIs: writing the GUI program from scratch or using the built-in GUIDE. GUIDE allows the user to graphically lay out the GUI and MATLAB generates the code for it automatically. However, to be able to understand and modify this code, it is important to understand the underlying programming concepts. Therefore, this section will concentrate on the programming methodology.

13.2.1 GUI Basics

A Figure Window is the parent of any GUI. Just calling the **figure** function will bring up a blank Figure Window; storing the handle in a variable allows us to manipulate its properties, as we have seen in Chapter 11.

```
>> f = figure
f =
  Figure (1) with properties:

      Number: 1
        Name: ''
       Color: [0.9400 0.9400 0.9400]
    Position: [440 378 560 420]
       Units: 'pixels'

  Show all properties
```

The position vector specifies `[left bottom width height]`. The first two numbers, the left and bottom, are the distance that the lower left corner of the figure box is from the lower left of the monitor screen (first from the left and then from the bottom). The last two are the width and height of the figure box itself. All of these are in the default units of pixels.

```
>> f.Visible
ans =
on
```

The 'Visible' property "on" means that the Figure Window can be seen. When creating a GUI, however, the normal procedure is to create the parent Figure Window, but make it invisible. Then, all user interface objects are added to it and properties are set. When everything has been completed the GUI is made visible.

If the figure is the only Figure Window that has been opened, then it is the current figure. Using **gcf** would be equivalent to *f* in that case. Recall that the parent of a Figure Window is the screen; this could be obtained with either *f.Parent* or **groot**.

Most user interface objects are created using the **uicontrol** function. Pairs of arguments are passed to the **uicontrol** function, consisting of the name of a property as a string and then its value. The default is that the object is created in the current figure; otherwise, a parent can be specified as in **uicontrol(parent,…)**. The 'Style' property defines the type of object, as a string. For example, 'text' is the Style of a static text box, which is normally used as a label for other objects in the GUI or for instructions.

The following example creates a GUI that just consists of a static text box in a Figure Window. The figure is first created, but made invisible. The color is white and it is given a position. Storing the handle of this figure in a variable allows the function to refer to it later on, to set properties, for example.

The **uicontrol** function is used to create a text box, position it (the vector specifies the [left bottom width height] within the Figure Window itself), and put a string in it. Note that the position is within the Figure Window, not within the screen.

A name is put on the top of the figure. The **movegui** function moves the GUI (the figure) to the center of the screen. Finally, when everything has been completed, the GUI is made visible.

simpleGui.m

```
function simpleGui
% simpleGui creates a simple GUI with just a static text box
% Format: simpleGui or simpleGui()

% Create the GUI but make it invisible for now while
%  it is being initialized
f = figure('Visible', 'off','color','white','Position',...
    [300, 400, 450,250]);
htext = uicontrol('Style','text','Position', ...
     [200,50, 100, 25], 'String','My First GUI string');

% Put a name on it and move to the center of the screen
% Note in older versions: set(f,'Name','Simple GUI')
f.Name = 'Simple GUI';
movegui(f,'center')

% Now the GUI is made visible
f.Visible = 'on';
end
```

The Figure Window shown in Fig. 13.15 will appear in the middle of the screen. The static text box requires no interaction with the user, so although this example shows some of the basics, it does not allow any graphical interface with the user.

When the 'Units' property of objects is set to 'Normalized,' this means that rather than specifying in pixels the position, it is done as a percentage. This allows the Figure Window to be resized without changing the way that the GUI appears. It also means that GUIs will look the same on different screen sizes. Normalized units for the figure means as a percentage of the screen, whereas Normalized units for a uicontrol object means as a percentage of the Figure Window. For example, the function *simpleGuiNormalized* is a version of the first GUI example that uses Normalized units:

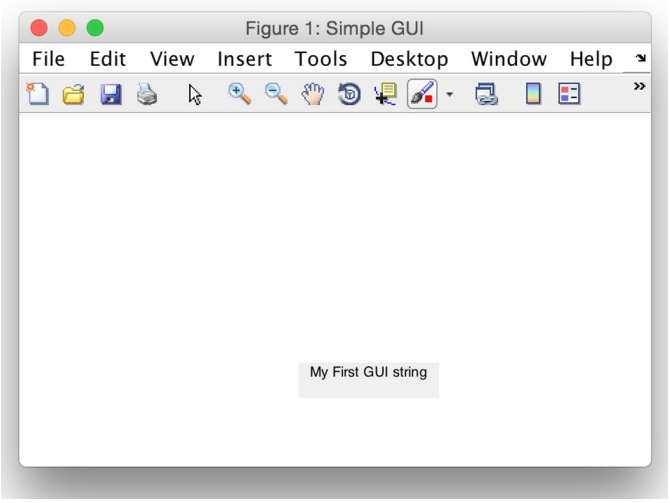


FIGURE 13.15
Simple GUI with a static text box.

simpleGuiNormalized.m

```
function simpleGuiNormalized
% simpleGuiNormalized creates a GUI with just a static text box
% Format: simpleGuiNormalized or simpleGuiNormalized()

% Create the GUI but make it invisible for now while
%  it is being initialized
f = figure('Visible', 'off','color','white','Units',...
 'Normalized', 'Position', [.25, .5, .35, .3]);
htext = uicontrol('Style','text','Units', 'Normalized', ...
  'Position', [.45, .2, .2, .1], ...
  'String','My First GUI string');
% Put a name on it and move to the center of the screen
f.Name = 'Simple GUI Normalized';
movegui(f,'center')

% Now the GUI is made visible
f.Visible = 'on';
end
```

For example, the first two numbers in the position vector for the figure [.25 .5 .35 .3] specifies that the lower left corner of the Figure Window is one quarter of the way from the left and half-way from the bottom of the screen. Of course, the figure is then moved to the middle of the screen, but the position vector is still useful in that it also specifies the width and height of the figure.

Since Normalized units are more general and more intuitive, for the most part they will be used for the remaining examples rather than pixels.

13.2.2 Text Boxes, Push Buttons, and Sliders

Now that we have seen the basic algorithm for a GUI, we will add user interaction.

In the next example, we will allow the user to enter a string in an editable text box and then the GUI will print the user's string in red. In this example there will be user interaction. First, the user must type in a string and, once this happens, the user's entry in the editable text box will no longer be shown, but, instead, the string that the user typed will be displayed in a larger red font in a static text box. When the user's action (which is called an ***event***) causes a response, what happens is that a ***callback function*** is called or invoked. The callback function is the part of the code in which the string is read in and then printed in a larger red font. This is sometimes called ***event-driven programming***: the event triggers an action.

The callback function must be in the path; one way to do this is to make it a nested function within the GUI function. The algorithm for this example is as follows.

- Create the Figure Window, but make it invisible.
- Make the color of the figure white, put a title on it, and move it to the center.
- Create a static text box with an instruction to enter a string.
- Create an editable text box.
  - The Style of this is 'edit.'
  - The callback function must be specified as the user's entry of a string necessitates a response (the function handle of the nested function is used for this).
- Make the GUI visible so that the user can see the instruction and type in a string.
- When the string is entered, the callback function *callbackfn* is called. Note that in the function header, there are two input arguments, *hObject* and *eventdata*. The input argument *hObject* refers to the handle of the **uicontrol** object that called it; *eventdata* stores in a structure information about the event that triggered the call to the callback function (this varies depending on the type of object and is not always used).
- The algorithm for the nested function *callbackfn* is
  - make the previous GUI objects invisible
  - get the string that the user typed (note that either *hObject* or the object handle name *huitext* can be used to refer to the object in which the string was entered)

- create a static text box to print the string in red with a larger font
- make this new object visible

guiWithEditbox.m

```
function guiWithEditbox
% guiWithEditbox has an editable text box
%   and a callback function that prints the user's
%   string in red
% Format: guiWithEditbox or guiWithEditbox()

% Create the GUI but make it invisible for now
f = figure('Visible', 'off','color','white','Units',...
    'Normalized','Position', [.25 .5 .4 .2]);
% Put a name on it and move it to the center of the screen
f.Name = 'GUI with editable text';
movegui(f,'center')
% Create two objects: a box where the user can type and
%   edit a string and also a text title for the edit box
hsttext = uicontrol('Style','text',...
     'BackgroundColor','white','Units','Normalized',...
     'Position',[.2 .6 .6 .2],...
     'String','Enter your string here');
huitext = uicontrol('Style','edit', 'Units',...
     'Normalized','Position',[.3 .3 .4 .2], ...
     'Callback',@callbackfn);
% Now the GUI is made visible
f.Visible = 'on';

     % Call back function
     function callbackfn(hObject,eventdata)
          % callbackfn is called by the 'Callback' property
          % in the editable text box
          set([hsttext huitext],'Visible','off');
          % Get the string that the user entered and print
          %    it in big red letters
          printstr = huitext.String;
          hstr = uicontrol('Style','text',...
               'BackgroundColor','white','Units',...
               'Normalized','Position',[.1 .3 .8 .4],...
               'String',printstr,...
               'ForegroundColor','Red','FontSize',30);
          hstr.Visible = 'on';
     end
end
```

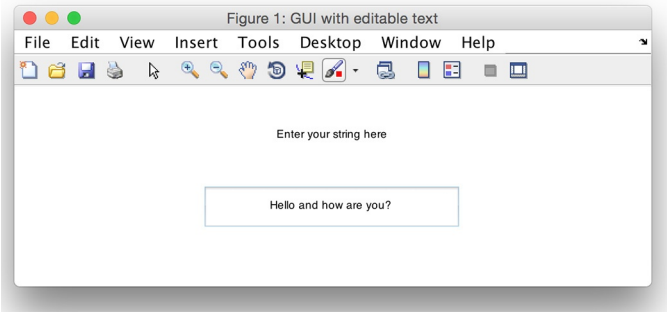


FIGURE 13.16
String entered by user in editable text box.

When the Figure Window is first made visible, the static text and the editable text box are shown. In this case, the user entered 'Hello and how are you?' Note that to enter the string, the user must first click the mouse in the editable text box. The string that was entered by the user is shown in Fig. 13.16.

After the user enters this string and hits the Enter key, the callback function is executed; the results are shown in Fig. 13.17. The callback function sets the Visible property to off for both of the original objects by referring to their handles. As the callback function is a nested function, the handle variables can be used. It then gets the string and writes it in a new static text box in red.

Now we will add a pushbutton to the GUI. This time, the user will enter a string, but the callback function will be invoked when the pushbutton is pushed.

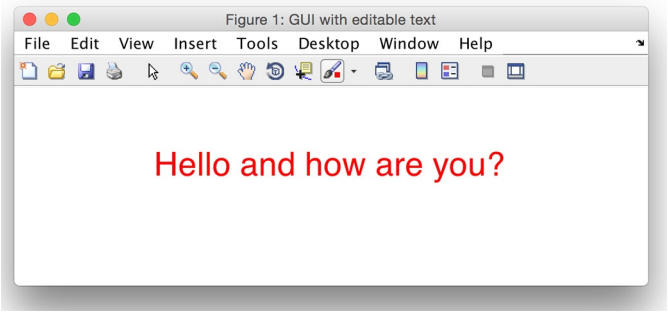


FIGURE 13.17
The result from the callback function.

guiWithPushbutton.m

```
function guiWithPushbutton
% guiWithPushbutton has an editable text box and a pushbutton
% Format: guiWithPushbutton or guiWithPushbutton()

% Create the GUI but make it invisible for now while
%  it is being initialized
f = figure('Visible', 'off','color','white','Units',...
   'Normalized', 'Position',[.25 .5 .5 .3]);
hsttext = ...
uicontrol('Style','text','BackgroundColor','white',...
    'Units','Normalized','Position',[.2 .7 .6 .2],...
    'String','Enter a string here, then push the button');
huitext = uicontrol('Style','edit','Units','Normalized',...
     'Position',[.3 .5 .4 .2]);
f.Name = 'GUI with pushbutton';
movegui(f,'center')

% Create a pushbutton that says "Push me!!"
hbutton = uicontrol('Style','pushbutton','String',...
     'Push me!!', 'Units','Normalized','Position',...
     [.6 .1 .3 .2], 'Callback',@callbackfn);

% Now the GUI is made visible
f.Visible = 'on';

% Call back function
     function callbackfn(hObject,eventdata)
          % callbackfn is called by the 'Callback' property
          % in the pushbutton
          set([hsttext huitext hbutton],'Visible','off');
          printstr = huitext.String;
          if isempty(printstr)
              printstr = 'Enter something next time!';
          end
          hstr = uicontrol('Style','text','BackgroundColor',...
              'white', 'Units', 'Normalized','Position',...
              [.1 .3 .8 .4], 'String', printstr, ...
              'ForegroundColor','Red','FontSize',30);
          hstr.Visible = 'on';
     end
end
```

In this case, the user types the string into the edit box. Hitting Enter, however, does not cause the callback function to be called; instead, the user must push the button with the mouse. The callback function is associated with the push-button object. So, pushing the button will bring up the string in a larger red font. The initial configuration with the pushbutton is shown in Fig. 13.18.

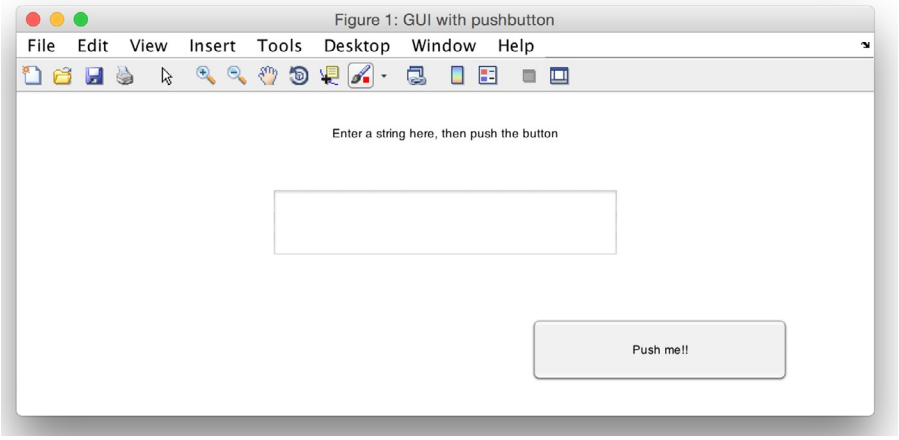


FIGURE 13.18
GUI with a push button.

PRACTICE 13.3

Create a GUI that will convert a length from inches to centimeters. The GUI should have an editable text box in which the user enters a length in inches, and a pushbutton that says "Convert me!" Pushing the button causes the GUI to calculate the length in centimeters and display that. The callback function that accomplishes this should leave all objects visible. That means the user can continue converting lengths until the Figure Window is closed. The GUI should display a default length to begin with (e.g., 1 in.). For example, calling the function might bring up the Figure Window shown in Fig. 13.19.

Then, when the user enters a length (e.g., 5.2 in.) and pushes the button, the Figure Window will show the new calculated length in centimeters (as seen in Fig. 13.20).

Another GUI object that can be created is a slider. The slider object has a numerical value, and can be controlled by either clicking on the arrows to move the value up or down, or by sliding the bar with the mouse. By default, the numerical value ranges from 0 to 1, but these values can be modified using the 'Min' and 'Max' properties.

The function *guiSlider* creates in a Figure Window a slider that has a minimum value of 0 and a maximum value of 5. It uses text boxes to show the minimum and maximum values, and also the current value of the slider.

guiSlider.m

```
function guiSlider
% guiSlider is a GUI with a slider
% Format: guiSlider or guiSlider()

f = figure('Visible', 'off','color','white','Units',...
    'Normalized', 'Position', [.25 .5 .4 .2]);

% Minimum and maximum values for slider
minval = 0;
maxval = 5;

% Create the slider object
slhan = uicontrol('Style','slider', 'Units', 'Normalized', ...
     'Position',[.3 .5 .4 .2], 'Min', minval, 'Max', maxval,...
     'SliderStep', [0.5 0.5], 'Callback', @callbackfn);
% Text boxes to show the minimum and maximum values
hmintext = uicontrol('Style','text','BackgroundColor',...
     'white', 'Units', 'Normalized',...
     'Position', [.1 .5 .1 .1], 'String', num2str(minval));
hmaxtext = uicontrol('Style', 'text','BackgroundColor',...
     'white', 'Units', 'Normalized',...
     'Position', [.8 .5 .1 .1], 'String', num2str(maxval));
% Text box to show the current value (off for now)
hsttext = uicontrol('Style','text','BackgroundColor','white',...
     'Units', 'Normalized',...
     'Position',[.4 .3 .2 .1],'Visible', 'off');

movegui(f,'center')
f.Name = 'Slider Example';
f.Visible = 'on';

% Call back function displays the current slider value
   function callbackfn(hObject,eventdata)
        % callbackfn is called by the 'Callback' property
        % in the slider
        num = slhan.Value;
        set(hsttext,'Visible','on','String',num2str(num))
    end
end
```

Fig. 13.21 shows the configuration once the user has moved the slider.

PRACTICE 13.4

Use the Help browser to find the property that controls the increment value on the slider, and modify the *guiSlider* function to move in increments of 0.5 when the arrow is used.

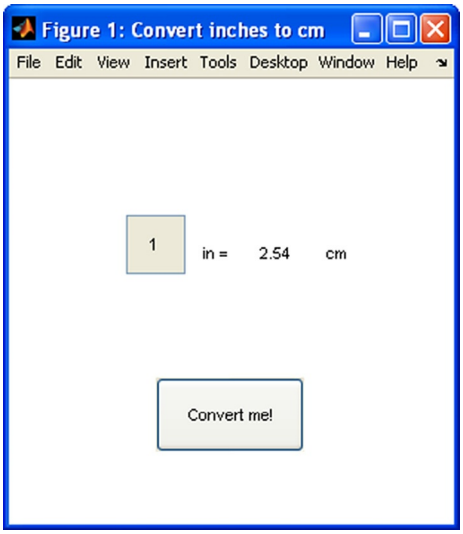


FIGURE 13.19
Length conversion GUI with push button.

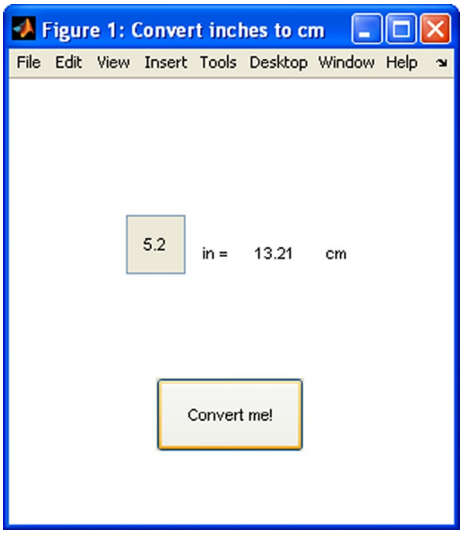


FIGURE 13.20
Result from conversion GUI.

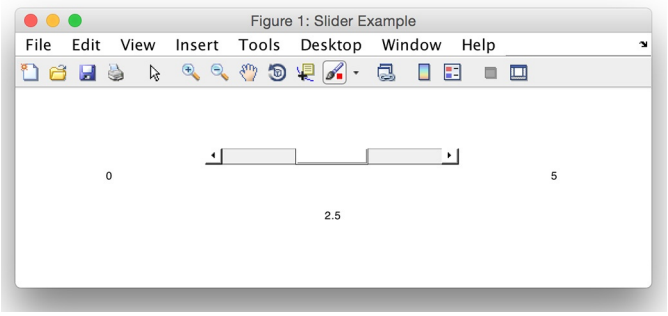


FIGURE 13.21
GUI with slider result shown.

GUI functions can have multiple callback functions. In the example *guiWithTwoPushbuttons*, there are two buttons that could be pushed (see Fig. 13.22). Each of them has a unique callback function associated with it. If the top button is pushed, its callback function prints red exclamation points (as shown in Fig. 13.23). If the bottom button is instead pushed, its callback function prints blue asterisks.

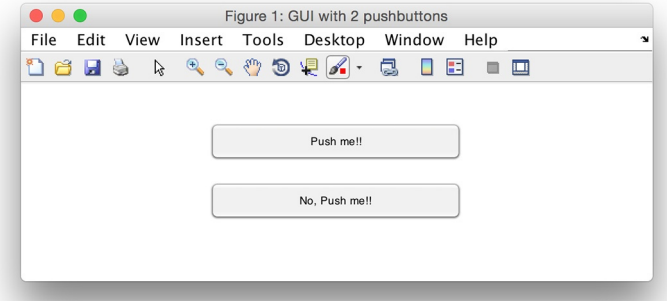


FIGURE 13.22
GUI with two pushbuttons and two callback functions.

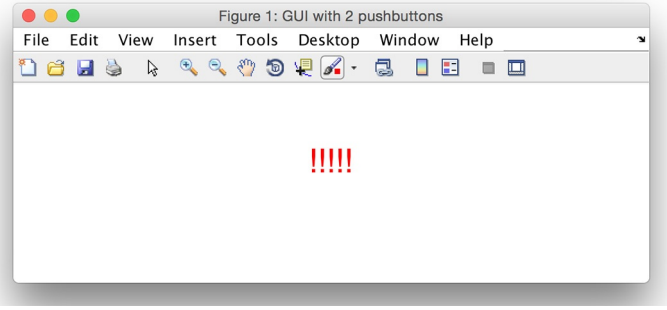


FIGURE 13.23
The result from the first callback function.

guiWithTwoPushbuttons.m

```
function guiWithTwoPushbuttons
% guiWithTwoPushbuttons has two pushbuttons, each
%    of which has a separate callback function
% Format: guiWithTwoPushbuttons

% Create the GUI but make it invisible for now while
%  it is being initialized
f = figure('Visible', 'off','color','white','Units',...
   'Normalized', 'Position', [.25 .5 .4 .2]);
f.Name = 'GUI with 2 pushbuttons';
movegui(f,'center')

% Create a pushbutton that says "Push me!!"
hbutton1 = uicontrol('Style','pushbutton','String',...
    'Push me!!', 'Units', 'Normalized',...
    'Position',[.3 .6 .4 .2], ...
    'Callback',@callbackfn1);

% Create a pushbutton that says "No, Push me!!"
hbutton2 = uicontrol('Style','pushbutton','String',...
    'No, Push me!!', 'Units', 'Normalized',...
    'Position',[.3 .3 .4 .2], ...
    'Callback',@callbackfn2);
% Now the GUI is made visible
f.Visible = 'on';

    % Call back function for first button
    function callbackfn1(hObject,eventdata)
       %  callbackfn is called by the 'Callback' property
       %  in the first pushbutton

        set([hbutton1 hbutton2],'Visible','off');
        hstr = uicontrol('Style','text',...
            'BackgroundColor', 'white', 'Units',...
            'Normalized','Position',[.4 .5 .2 .2],...
            'String','!!!!!', ...
            'ForegroundColor','Red','FontSize',30);
        hstr.Visible = 'on';
    end

 % Call back function for second button
 function callbackfn2(hObject,eventdata)
   % callbackfn is called by the 'Callback' property
   % in the second pushbutton

    set([hbutton1 hbutton2],'Visible','off');
    hstr = uicontrol('Style','text',...
        'BackgroundColor','white', 'Units', ...
        'Normalized', 'Position',[.4 .5 .2 .2],...
        'String','*****', ...
        'ForegroundColor','Blue','FontSize',30);
    hstr.Visible = 'on';
 end

end
```

If the first button is pushed, the first callback function is called, which would produce the result in Fig. 13.23.

It is also possible to have one callback function invoked, or called, by multiple objects. In the example with two pushbuttons, instead of having one callback function associated with each pushbutton, we could have just one callback function. In that case, the value of the input argument *hObject* would be checked to determine which object called it. A modified GUI to accomplish this is shown next.

guiWithTwoPushbuttonsii.m

```
function guiWithTwoPushbuttonsii
% guiWithTwoPushbuttonsii has two pushbuttons
%    but just one callback function
% Format: guiWithTwoPushbuttonsii

% Create the GUI but make it invisible for now while
%  it is being initialized
f = figure('Visible', 'off','color','white','Units',...
    'Normalized', 'Position', [.25 .5 .4 .2]);
f.Name = 'GUI with 2 pushbuttons';
movegui(f,'center')

% Create a pushbutton that says "Push me!!"
hbutton1 = uicontrol('Style','pushbutton','String',...
     'Push me!!', 'Units', 'Normalized',...
     'Position',[.3 .6 .4 .2], ...
     'Callback',@callbackfn);

% Create a pushbutton that says "No, Push me!!"
hbutton2 = uicontrol('Style','pushbutton','String',...
     'No, Push me!!', 'Units', 'Normalized',...
     'Position',[.3 .3 .4 .2], ...
     'Callback',@callbackfn);
% Now the GUI is made visible
f.Visible = 'on';

     % Call back function for both buttons
     function callbackfn(hObject,eventdata)
         % callbackfn is called by the 'Callback' property

          set([hbutton1 hbutton2],'Visible','off');
          hstr = uicontrol('Style','text',...
               'BackgroundColor', 'white', 'Units',...
               'Normalized','Position',[.4 .5 .2 .2],...
               'FontSize',30);
          if hObject == hbutton1
               hstr.String = '!!!!!!';
               hstr.ForegroundColor = 'Red';
          else
               hstr.String = '*****';
               hstr.ForegroundColor = 'Blue';
          end
          hstr.Visible = 'on';
     end

end
```

In this example, the value of the input argument *hObject* was used by the callback function. The second input argument, *eventdata*, was not used, however. Whenever any callback function is called, both of these arguments are automatically passed to the callback function. If the callback function is not going to use one or both of them, the tilde can be put in the function header instead of the name of the input argument. In the previous example, the header could have been:

```
function callbackfn(hObject,~)
```

13.2.3 Plots and Images in GUIs

Plots and images can be imbedded in a GUI. The next example *guiSliderPlot* shows a plot of **sin(x)** from 0 to the value of a slider bar. The axes are positioned within the Figure Window using the **axes** function, and then, when the slider is moved the callback function plots. Note that the default Units property for an **axes** object is Normalized.

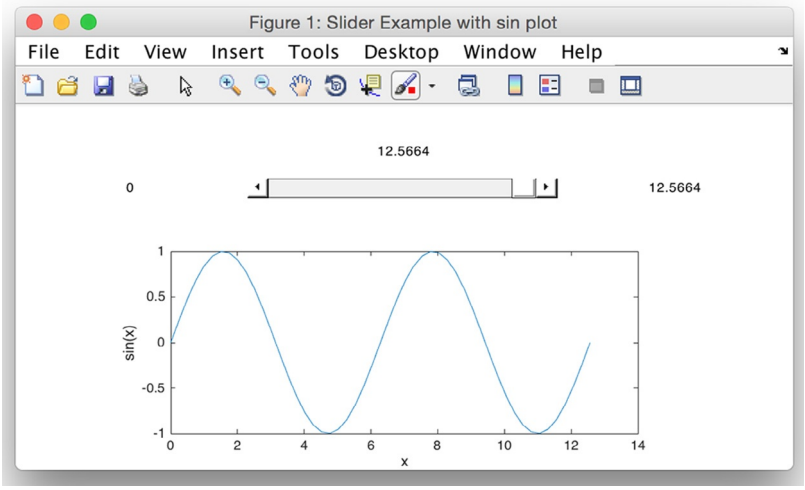


FIGURE 13.24
Plot shown in a GUI Figure Window.

guiSliderPlot.m

```
function guiSliderPlot
% guiSliderPlot has a slider
% It plots sin(x) from 0 to the value of the slider
% Format: guisliderPlot

f = figure('Visible', 'off', 'Units', 'Normalized',...
    'Position', [.2 .5 .4 .3], 'Color', 'white');

% Minimum and maximum values for slider
minval = 0;
maxval = 4*pi;

% Create the slider object
slhan = uicontrol('Style','slider','Units','Normalized',...
    'Position',[.3 .7 .4 .1], ...
    'Min', minval, 'Max', maxval,'Callback', @callbackfn);
% Text boxes to show the min and max values and slider value
hmintext = uicontrol('Style','text','BackgroundColor', 'white', ...
    'Units','Normalized','Position', [.1 .7 .1 .1],...
    'String', num2str(minval));
hmaxtext = uicontrol('Style','text', 'BackgroundColor', 'white',...
    'Units', 'Normalized','Position', [.8 .7 .1 .1], ...
    'String', num2str(maxval));
hsttext = uicontrol('Style','text','BackgroundColor', 'white',...
    'Units','Normalized', 'Position', [.45 .8 .1 .1],...
    'Visible','off');
% Create axes handle for plot
axhan = axes('Position', [.2 .1 .6 .5]);

f.Name = 'Slider Example with sin plot';
movegui(f,'center')
f.Visible = 'on';

% Call back function displays the current slider value & plots sin
   function callbackfn(~,~)
       % callbackfn is called by the 'Callback' property
       % in the slider
       num = slhan.Value;
       set(hsttext,'Visible','on','String',num2str(num))
       x = 0:num/50:num;
       y = sin(x);
       plot(x,y)
       xlabel('x')
       ylabel('sin(x)')
   end
end
```

Fig. 13.24 shows the configuration of the window, with the slider bar, static text boxes to the left and right showing the minimum and maximum values, and

the axes positioned underneath. After the slider bar is moved, the callback function plots **sin(x)** from 0 to the position of the slider bar.

Images can also be placed in GUIs, again using **axes** to locate the image. In a variation of the previous example, the next example displays an image and uses a slider to vary the brightness of the image. The result is shown in Fig. 13.25.

guiSliderImage.m

```
function guiSliderImage
% guiSliderImage has a slider
% Displays an image; slider dims it
% Format: guisliderImage

f = figure('Visible', 'off', 'Units', 'Normalized',...
    'Position', [.2 .5 .4 .3], 'Color', 'white');
% Minimum and maximum values for slider
minval = 0;
maxval = 1;
% Create the slider object
slhan = uicontrol('Style','slider','Units','Normalized',...
    'Position',[.3 .7 .4 .1], ...
    'Min', minval, 'Max', maxval,'Callback', @callbackfn);
% Text boxes to show the min and max values and slider value
hmintext = uicontrol('Style','text','BackgroundColor', 'white', ...
    'Units','Normalized','Position', [.1 .7 .1 .1],...
    'String', num2str(minval));
hmaxtext = uicontrol('Style','text', 'BackgroundColor', 'white',...
    'Units', 'Normalized','Position', [.8 .7 .1 .1], ...
    'String', num2str(maxval));
hsttext = uicontrol('Style','text','BackgroundColor', 'white',...
    'Units','Normalized', 'Position', [.45 .8 .1 .1],...
    'Visible','off');
% Create axes handle for plot
axhan = axes('Position', [.2 .1 .6 .5]);

f.Name = 'Slider Example with an image';
movegui(f,'center')
f.Visible = 'on';

% Call back function displays the current slider value & image
   function callbackfn(~,~)
        % callbackfn is called by the 'Callback' property
        % in the slider
        num = slhan.Value;
        set(hsttext,'Visible','on','String',num2str(num))

        myimage1 = imread('snowyporch.JPG');
        dimmer = num * myimage1;
        image(dimmer)
    end
end
```

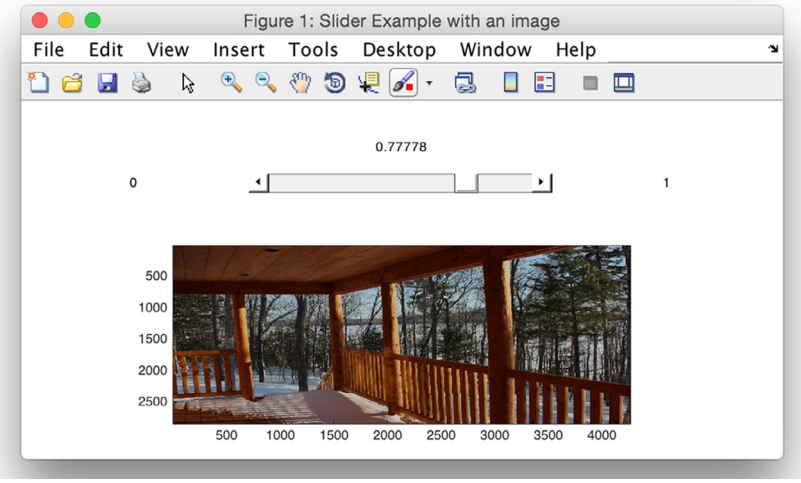


FIGURE 13.25
GUI with an image and slider for brightness.

13.2.4 Button Groups

This section illustrates radio buttons, and grouping objects together; in this case, in a button group.

The next GUI presents the user with a choice of colors using two radio buttons, only one of which can be chosen at any given time. The GUI prints a string to the right of the radio buttons, in the chosen color.

The function **uibuttongroup** creates a mechanism for grouping together the buttons. As only one button can be chosen at a time, there is a type of callback function called 'SelectionChangedFcn' that is called when a button is chosen (note: prior to R2014b this was named 'SelectionChangeFcn').

This function gets from the button group which button is chosen with the 'SelectedObject' property. It then chooses the color based on this. This property is set initially to the empty vector, so that neither button is selected; the default is that the first button would be selected.

guiWithButtongroup.m

```
function guiWithButtongroup
% guiWithButtongroup has a button group with 2 radio buttons
% Format: guiWithButtongroup

% Create the GUI but make it invisible for now while
%  it is being initialized
f = figure('Visible', 'off','color','white','Units',...
    'Normalized', 'Position',[.2 .5 .4 .3]);

%   Create a button group
grouph = uibuttongroup('Parent',f,'Units','Normalized',...
    'Position',[.2 .5 .4 .4], 'Title','Choose Color',...
    'SelectionChangedFcn',@whattodo);

%   Put two radio buttons in the group
toph = uicontrol(grouph,'Style','radiobutton',...
    'String','Blue','Units','Normalized',...
    'Position', [.2 .7 .4 .2]);

both = uicontrol(grouph, 'Style','radiobutton',...
    'String','Green','Units','Normalized',...
    'Position',[.2 .4 .4 .2]);

%   Put a static text box to the right
texth = uicontrol('Style','text','Units','Normalized',...
    'Position',[.6 .5 .3 .3],'String','Hello',...
    'Visible','off','BackgroundColor','white');

grouph.SelectedObject = []; % No button selected yet
f.Name = 'GUI with button group';
movegui(f,'center')

% Now the GUI is made visible
f.Visible = 'on';

    function whattodo(~, ~)
    % whattodo is called by the 'SelectionChangedFcn' property
    % in the button group

    which = get(grouph,'SelectedObject');

    if which == toph
        texth.ForegroundColor = 'blue';
    else
        texth.ForegroundColor = 'green';
    end

    texth.Visible = 'on';
    end

end
```

Fig. 13.26 shows the initial configuration of the GUI: the button group is in place, as are the buttons (but neither is chosen).

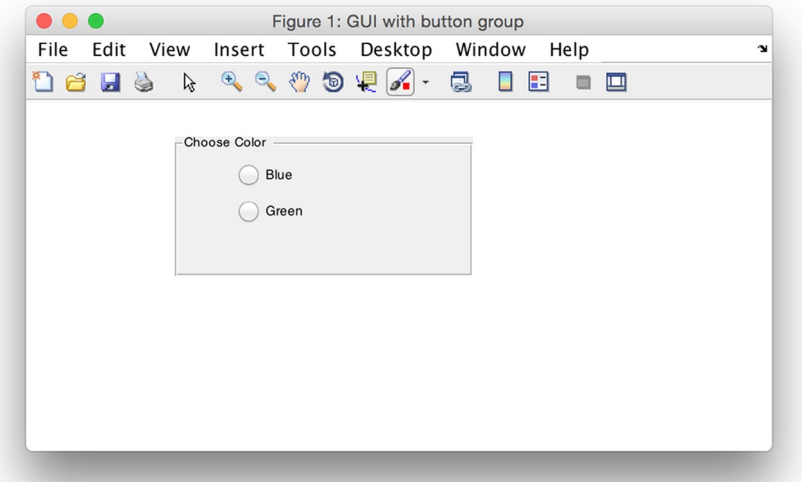


FIGURE 13.26
Button group with radio buttons.

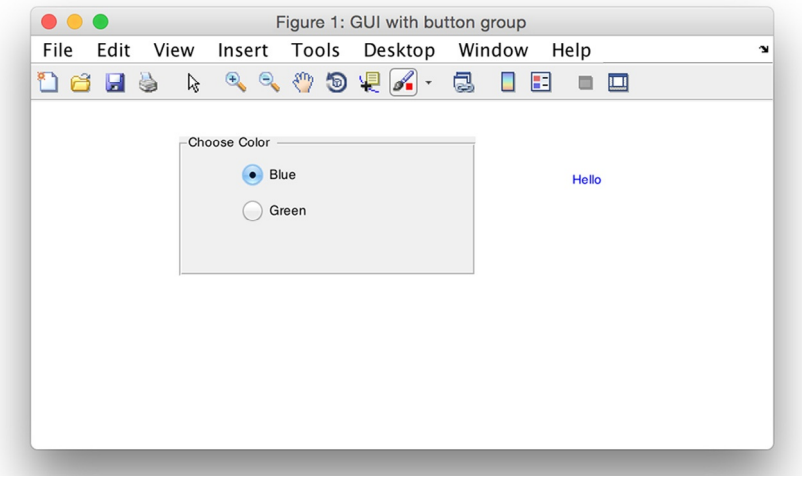


FIGURE 13.27
Button group: choice of color for string.

Once a radio button has been chosen, the *whattodo* function chooses the color for the string, which is printed in a static text box on the right, as shown in Fig. 13.27.

13.3 GUIDE AND APP DESIGNER

Current versions of MATLAB, as of R2016a, have two development environments in which GUIs can be graphically created and the code will be automatically generated. GUIDE creates code for GUIs that is procedural and similar to

the examples shown in the previous section. The App Designer uses similar functions, but is all object-based. GUIDE will be briefly introduced to show that the programming techniques covered in the previous section can be used to modify code generated by GUIDE. More complete coverage is given in this section to App Designer, which creates app classes derived from a superclass, and reinforces the OOP concepts covered in Chapter 11.

13.3.1 GUIDE

In order to get into GUIDE, the command **guide** can be entered in the Command Window.

```
>> guide
```

This brings up a dialog box in which you should choose "Blank GUI (Default)" and then click the "OK" button. This brings up a GUIDE layout window with icons for objects on the left that can be dragged into the grid, to graphically lay out a GUI. However, without creating anything, we will see the basic code that GUIDE creates for a blank GUI. Clicking on the green arrow "Run" button prompts for a name and then saves two separate files: a figure file (.fig) and a function (.m) file. For example, if we enter the name "guideGUI1," the files guideGUI1.fig and guideGUI1.m are created. The function file contains the following:

```
function varargout = guideGUI1(varargin)
% GUIDEGUI1 MATLAB code for guideGUI1.fig
%       GUIDEGUI1, by itself, creates a new GUIDEGUI1 or raises the existing
%       singleton*.
%
%       H = GUIDEGUI1 returns the handle to a new GUIDEGUI1 or the handle to
%       the existing singleton*.
%
%       GUIDEGUI1('CALLBACK',hObject,eventData,handles,...) calls the
%       local function named CALLBACK in GUIDEGUI1.M with the given input
%       arguments.
%
%       GUIDEGUI1('Property','Value',...) creates a new GUIDEGUI1 or
%       raises the existing singleton*. Starting from the left, property
%       value pairs are applied to the GUI before guideGUI1_OpeningFcn gets
%       called. An unrecognized property name or invalid value makes property
%       application stop. All inputs are passed to guideGUI1_OpeningFcn
%       via varargin.
%
%       *See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one
%       instance to run (singleton)".
%
% See also: GUIDE, GUIDATA, GUIHANDLES
```

```
% Edit the above text to modify the response to help guideGUI1

% Last Modified by GUIDE v2.5 29-Jan-2016 08:55:11

% Begin initialization code - DO NOT EDIT
gui_Singleton = 1;
gui_State = struct('gui_Name',       mfilename, ...
                   'gui_Singleton',  gui_Singleton, ...
                   'gui_OpeningFcn', @guideGUI1_OpeningFcn, ...
                   'gui_OutputFcn',  @guideGUI1_OutputFcn, ...
                   'gui_LayoutFcn',  [] , ...
                   'gui_Callback',   []);
if nargin && ischar(varargin{1})
    gui_State.gui_Callback = str2func(varargin{1});
end

if nargout
    [varargout{1:nargout}] = gui_mainfcn(gui_State, varargin{:});
else
    gui_mainfcn(gui_State, varargin{:});
end
% End initialization code - DO NOT EDIT

% --- Executes just before guideGUI1 is made visible.
function guideGUI1_OpeningFcn(hObject, eventdata, handles, varargin)
% This function has no output args, see OutputFcn.
% hObject  handle to figure
% eventdata reserved - to be defined in a future version of MATLAB
% handles  structure with handles and user data (see GUIDATA)
% varargin command line arguments to guideGUI1 (see VARARGIN)

% Choose default command line output for guideGUI1
handles.output = hObject;

% Update handles structure
guidata(hObject, handles);

% UIWAIT makes guideGUI1 wait for user response (see UIRESUME)
% uiwait(handles.figure1);

% --- Outputs from this function are returned to the command line.
function varargout = guideGUI1_OutputFcn(hObject, eventdata, handles)
% varargout cell array for returning output args (see VARARGOUT);
% hObject  handle to figure
% eventdata reserved - to be defined in a future version of MATLAB
% handles  structure with handles and user data (see GUIDATA)

% Get default command line output from handles structure
varargout{1} = handles.output;
```

The *guideGUI1* function creates a structure that stores some properties, as well as functions *guideGUI1_OpeningFcn* and *guideGUI1_OutputFcn* which execute

when the GUI is made visible and when the *guideGUI1* function finishes executing, respectively.

If, instead of creating a blank GUI, we dragged a static text box and an editable text box into the GUIDE layout, the code is similar to the previous example. However, it also creates a callback function stub and a create object function stub for the editable text box. Fig. 13.28 shows the object icons on the left, and part of the grid layout into which we have dragged a static text box and an editable text box, without resizing or changing the labels.

```
function edit1_Callback(hObject, eventdata, handles)
% hObject    handle to edit1 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of edit1 as text
%        str2double(get(hObject,'String')) returns contents of edit1 as a
double


% - - - Executes during object creation, after setting all properties.
function edit1_CreateFcn(hObject, eventdata, handles)
% hObject    handle to edit1 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),
get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end
```

The callback function *edit1_Callback* does not do anything by default, but it does contain a hint that since this is an editable text box, a reasonable thing to do would be to get the string from the box and then do something with it. At this point, it is possible to edit the code directly using the programming techniques from the previous section, or make modifications using GUIDE. The GUIDE documentation explains its features.

13.3.2 App Designer

The App Designer is similar to GUIDE in that it allows the user to graphically layout objects, which are called ***components***. However, there are more types of components than GUIDE has, and the code that is generated is all object-based. Also, App Designer creates UI Figure windows, whereas GUIDE uses Figure Windows.

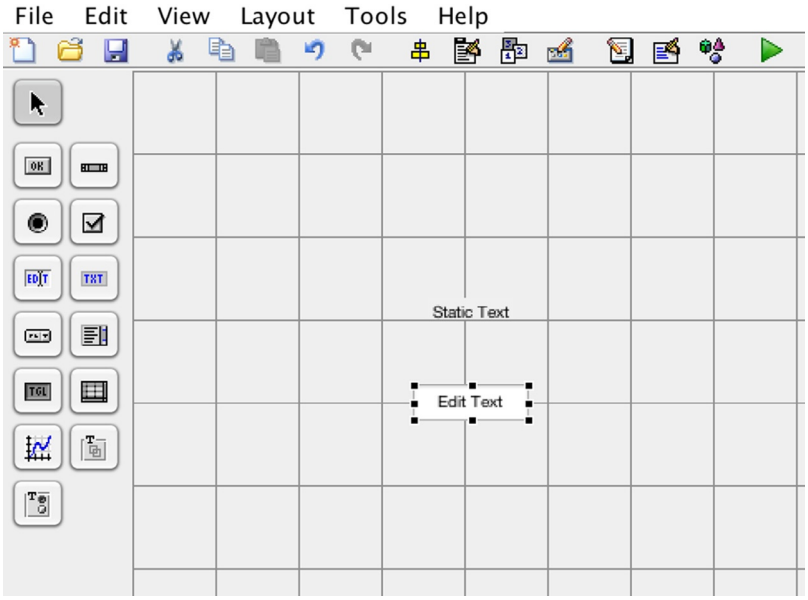


FIGURE 13.28
GUIDE layout of text boxes.

13.3.2.1 Intro to App Designer

Typing the command **appdesigner** in the Command Window will bring up the App Designer. Fig. 13.29 shows the App Designer; in the middle is the blank layout under "Design View." The Component Library on the left shows the icons of the components that can be created. On the right are the Component Browser and Component Properties Windows. In the Component Browser, the default name for the blank figure, *app.UIFigure*, can be seen.

```
>> appdesigner
```

Fig. 13.30 shows the remainder of the Component Library, including the Container and Instrumentation components.

Clicking on "Code View" instead of "Design View" shows the code that has been created for the blank app. App Designer creates a class named *App1* that is derived from a MATLAB apps superclass called **matlab.apps.AppBase**. The properties will consist of all of the components; for now, there is just one property, which is the UI Figure Window. There are also two private methods blocks and one public methods block.

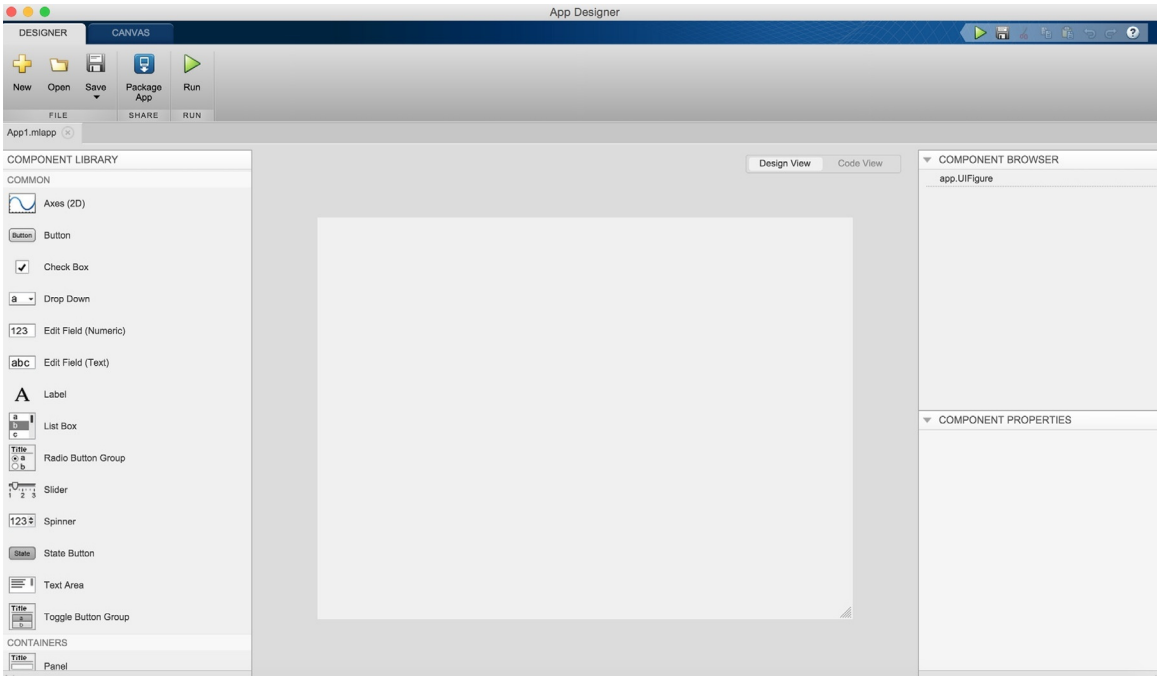


FIGURE 13.29
App Designer Layout.

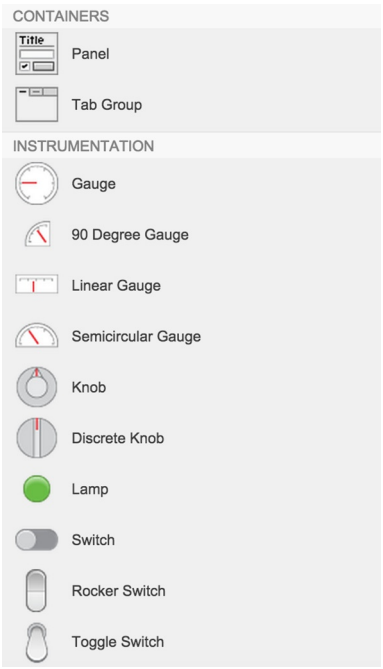


FIGURE 13.30
Bottom of Component Library.

```
classdef App1 < matlab.apps.AppBase
    % Properties that correspond to app components
    properties (Access = public)
        UIFigure matlab.ui.Figure % UI Figure
    end

    methods (Access = private)

        % Code that executes after component creation
        function startupFcn(app)

        end

    end

    % App initialization and construction
    methods (Access = private)

        % Create UIFigure and components
        function createComponents(app)

            % Create UIFigure
            app.UIFigure = uifigure;
            app.UIFigure.Position = [100 100 640 480];
            app.UIFigure.Name = 'UI Figure';
            setAutoResize(app, app.UIFigure, true)
        end
    end

    methods (Access = public)

        % Construct app
        function app = App1()

            % Create and configure components
            createComponents(app)

            % Register the app with App Designer
            registerApp(app, app.UIFigure)

            % Execute the startup function
            runStartupFcn(app, @startupFcn)

            if nargout == 0
                clear app
            end
        end

        % Code that executes before app deletion
        function delete(app)
            % Delete UIFigure when app is deleted
            delete(app.UIFigure)
        end
    end
end
```

The methods are used for all of the app functions, including eventually callback functions. For now, with just a blank UI Figure Window, the public methods include the constructor function *App1*, and a function *delete* that deletes the figure when the app is deleted. The constructor calls a private function *createComponents*, which creates a UI Figure using the **uifigure** function. It then uses the dot notation to set properties for the *app.UIFigure*, including its *Name* and *Position*. Note that the dot notation is used twice here, as in *app.UIFigure.Name*.

The function *startupFcn* executes when components are created. It is called by a function in the constructor. Within the Code View, most of the window is grey, which means that the code cannot be modified. However, within the *startupFcn* function there is a white box, which means that code can be inserted in that function. Fig. 13.31 shows the *startupFcn* function in Code View.

We have seen the code that is generated with just a blank canvas. In the Design View of the App Designer environment, components can be dragged from the Component Library onto the blank layout. When this is done, the code will be updated to create a new property for each component, and to initialize some of its properties (for example, the Position property will be based on the location to which the component was dragged). Properties can then be modified in the Component Properties Window to the right of the layout.

For example, dragging a Label (generically named 'Label'!) into the design area causes the Design View to look like Fig. 13.32.

Clicking on the label itself in the Design View will change "Component Properties" to "Label Properties." Properties such as the text on the label, its justification, font name, font size, and so forth can then be modified from the Label Properties window. Each of these will modify the code that is created.

Assuming that no properties have been modified, and there is just a generic label in the layout as shown in Fig. 13.32, most of the code remains the same as with just a blank UI Figure Window. Choosing Code View will show that the class *App1* now has a new property Label, and the *createComponents* method creates a Label using the **uilabel** function and gives it a Position. The added code is shown in bold here (not in App Designer itself).

```
 8      methods (Access = private)
 9
10          % Code that executes after component creation
11          function startupFcn(app)
12
13          end
14      end
15
```

FIGURE 13.31
Editable portion of default app code.

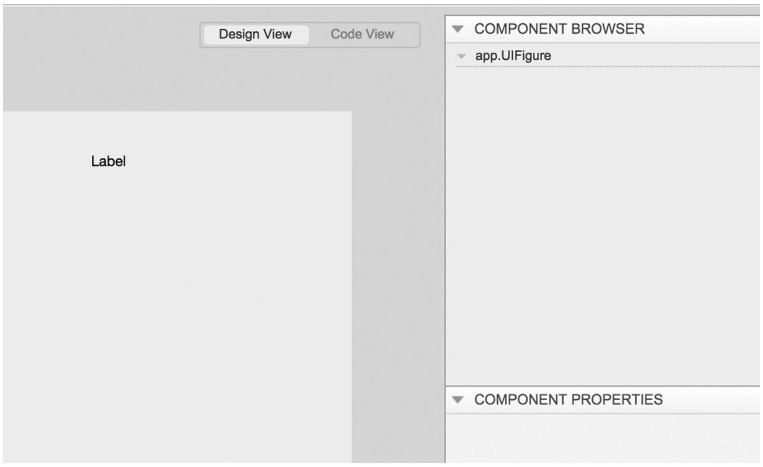


FIGURE 13.32
Simple Label in app.

```
classdef App1 < matlab.apps.AppBase
% Properties that correspond to app components
properties (Access = public)
    UIFigure matlab.ui.Figure % UI Figure
    Label   matlab.ui.control.Label % Label
end

% Create UIFigure and components
    function createComponents(app)
        % Create UIFigure
        app.UIFigure = uifigure;
        app.UIFigure.Position = [100 100 640 480];
        app.UIFigure.Name = 'UI Figure';
        setAutoResize(app, app.UIFigure, true)
        % Create Label
        app.Label = uilabel(app.UIFigure);
        app.Label.Position = [421 431 30 15];
    end
end
```

Modifying a property, such as changing the text of the label in the Label Properties window to be "Hello" instead of "Label" will modify the code in *createComponents* to:

```
% Create Label
app.Label = uilabel(app.UIFigure);
app.Label.Position = [421 431 30 15];
app.Label.Text = 'Hello';
```

It is also possible programmatically to modify properties such as *app.Label.Text* in the *startupFcn* function, but it is preferable and easier to do this from the Design View.

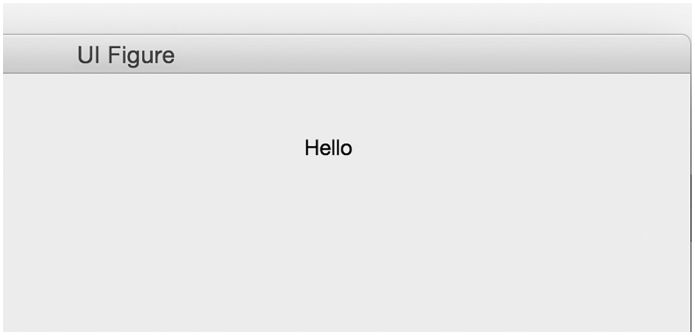


FIGURE 13.33
Text of label in UI Figure Window.

To run the app, click on the green Run arrow. This brings up a dialog box that asks for the name of the file that will be created; the default name that shows is "App1.mlapp." Changing the name to "HelloLabel.mlapp" will create a file with this name and bring up a UI Figure Window. The upper right portion of this is shown in Fig. 13.33.

Note that the App Designer creates one file with the extension '.mlapp,' unlike GUIDE, which creates two files: a .m file and a .fig file.

While this example shows the basics of App Designer and the object-based code that it creates, it did not involve any callbacks.

13.3.2.2 UI Figure Functions

As we have seen in Section 13.2, traditional GUIs create different types of objects by specifying the Style property in the **uicontrol** function. For example, a slider is created with **uicontrol('Style', 'slider')**. By contrast, App Designer uses separate functions to create the different component types; for example, a slider is created with the **uislider** function. In the previous section, we saw two of these functions: **uifigure**, which creates the UI Figure Window, and **uilabel**, which creates a static text box.

All of these functions can be called directly from the Command Window or from any script or function. For example, the following code will bring up the UI Figure Window shown in Fig. 13.34.

```
>> uif = uifigure;
>> uif.Name = 'Simple UI Fig Slider';
>> uislid = uislider(uif);
```

By creating a UI Figure Window first, and then passing its handle to the **uislider** function, the slider is put inside that UI Figure Window (otherwise, a new UI



FIGURE 13.34
Simple slider in UI Figure Window.

Figure Window would be created for the slider). The Value property of the slider can be inspected, e.g.,

```
>> uislid.Value
ans =
     0
```

Of course, to do anything with the value of the slider would require more complicated code with callback functions. For sliders, there are two:

'ValueChangedFcn': executes when the slider value has been changed
'ValueChangingFcn': executes as the slider is being moved

For example, the function *uislideruilabel* creates a UI Figure Window with a slider and a label. When the slider has been moved, the 'ValueChangedFcn' callback is called, which puts the value of the slider in the label as seen in Fig. 13.35. Many of the concepts here are similar to those used in GUI callbacks. The function *uislideruilabel* is a function that has a nested callback function, *whatslid*. Since the *whatslid* function is nested, the variable scope is such that the variables *uilab* and *uislid* can be used within the callback function without passing them. The callback function has two input arguments representing the source of the callback (in this case *uislid*) and the eventdata, just like GUI callback functions.

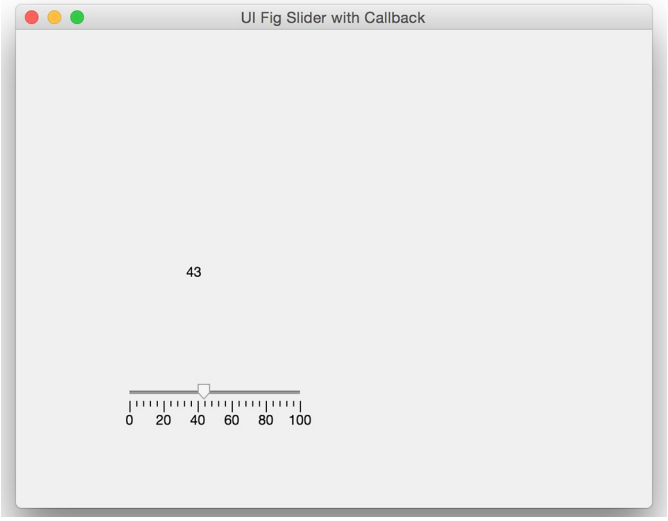


FIGURE 13.35
Slider with label example.

uislideruilabel.m

```
function uislideruilabel
  uif = uifigure;
  uif.Name = 'UI Fig Slider with Callback';
  uislid = uislider(uif);
  uilab = uilabel(uif,'Position',[150 200 30 15]);
  uilab.Text = '0';

  uislid.ValueChangedFcn = @whatslid;

  function whatslid(source,event)
  uilab.Text = num2str(round(uislid.Value));
  end
  end
```

The list of UI functions that create object components can be found in the App Designer documentation. The documentation for each function describes the properties that can be used for each, including all of the possible callback functions.

13.3.2.3 App Designer Example

An advantage of App Designer is that it will create much of the code automatically, and since it is based on objects, the private properties and methods protect the apps that are created.

As an example of an app that uses a callback, an app will be created that is similar to the previous example in which there is a slider and a label that shows the value of the slider. Since it makes sense to label many components, such as sliders, App Designer automatically creates a label when the component is dragged into the design area (unless the Ctrl key is pressed while dragging the component). By default the label is to the left of the slider. In Design View, since there are two components, the slider and its label, the properties window is initially labeled "Multiple Components Properties."

The Code View shows that three properties have been created:

```
properties (Access = public)
    UIFigure    matlab.ui.Figure          % UI Figure
    LabelSlider matlab.ui.control.Label   % Slider
    Slider      matlab.ui.control.Slider  % [0 100]
end
```

The constructor and other methods are similar to what was seen in a previous example. Unlike GUIDE, App Designer does not automatically create callback function stubs. Instead, to create a callback, it is necessary to right-click on the component in Design View. Selecting 'Callbacks' brings up two possible callback functions. For example, enter the name "SliderValueChanged" for the ValueChangedFcn. This creates a new function, seen in Code View:

```
% Slider value changed function
function SliderValueChanged(app)
    value = app.Slider.Value;

end
```

In the function, an example is given that shows how to reference the Value property of the slider, but the function is editable. Modifying the function body as follows will put the value of the slider in the label.

```
function SliderValueChanged(app)
    app.LabelSlider.Text = ...
          num2str(round(app.Slider.Value));
end
```

In Design View, clicking on either the slider or its label will change the Properties window. For example, clicking on the label allows one to change the initial label of "Slider" to "0," which is the initial value of the slider.

To change properties of any of the objects, one can click on the object name in the Component Browser. For example, clicking on "app.UIFigure" in the Component Browser then shows "UI Figure Properties." That allows us to change the title of the UI Figure Window, for example to "App with Slider."

When the app is executed, the UI Figure window seen in Fig. 13.36 is created.

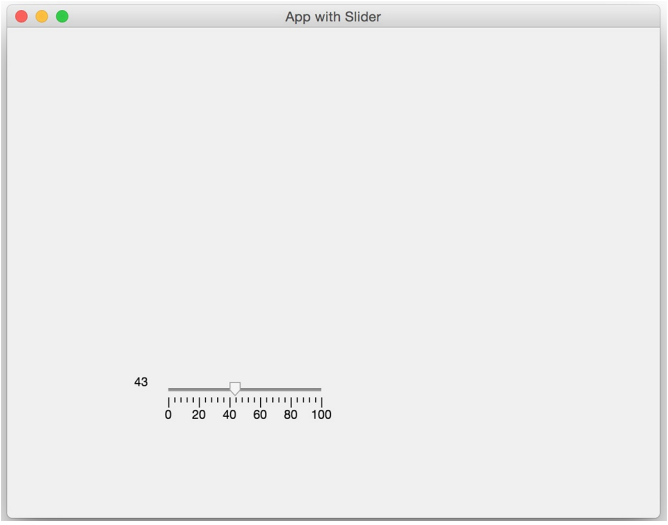


FIGURE 13.36
App with slider and label.

Putting plots into apps uses a method that is very similar to GUIs: it is necessary to first drag axes into the design layout, and then any plot will be placed within the current axes.

13.4 SOUND FILES

A ***sound wave*** is an example of a continuous signal that can be sampled to result in a discrete signal. In this case, sound waves traveling through the air are recorded as a set of measurements that can then be used to reconstruct the original sound signal, as closely as possible. The sampling rate, or ***sampling frequency***, is the number of samples taken per time unit, for example per second. Sound signals are usually measured in Hertz (Hz).

In MATLAB, the discrete sound signal is represented by a vector and the frequency is measured in Hertz. MATLAB has several MAT-files that store for various sounds the signal vector in a variable *y* and the frequency in a variable *Fs*. These MAT-files include **chirp**, **gong**, **laughter**, **splat**, **train**, and **handel**. There is a built-in function, **sound**, that will send a sound signal to an output device such as speakers.

The function call:

```
>> sound(y,Fs)
```

will play the sound represented by the vector *y* at the frequency *Fs*. For example, to hear a gong, load the variables from the MAT-file and then play the sound using the **sound** function:

```
>> load gong
>> sound(y,Fs)
```

Sound is a wave; the amplitudes are what are stored in the sound signal variable *y*. These are supposed to be in the range from −1 to 1. The **plot** function can be used to display the data. For example, the following script creates a **subplot** that displays the signals from **chirp** and from **train**, as shown in Fig. 13.37.

chirptrain.m

```
% Display the sound signals from chirp and train
subplot(2,1,1)
load chirp
plot(y)
ylabel('Amplitude')
title('Chirp')

subplot(2,1,2)
load train
plot(y)
ylabel('Amplitude')
title('Train')
```

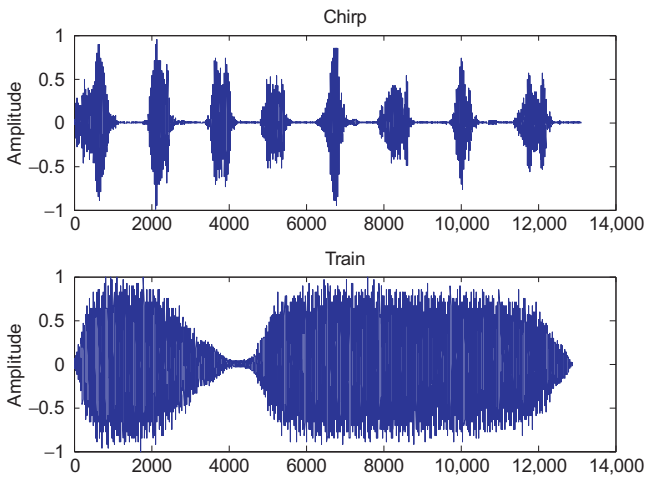


FIGURE 13.37
Amplitudes from **chirp** and **train**.

The first argument to the **sound** function can be an $n \times 2$ matrix for stereo sound. Also, the second argument can be omitted when calling the **sound** function, in which case the default sample frequency of 8192 Hz is used. This is the frequency stored in the built-in sound MAT-files.

```
>> load train
Fs
Fs =
        8192
```

PRACTICE 13.5

If you have speakers try loading one of the sound MAT-files and use the **sound** function to play the sound. Then, change the frequency; for instance, multiply the variable *Fs* by 2 and by 0.5, and play these sounds again.

```
>> load train
>> sound(y, Fs)
>> sound(y, Fs*2)
>> sound(y, Fs*.5)
```

Explore Other Interesting Features

Several ***audio file formats*** are used in industry on different computer platforms. Audio files with the extension ".au" were developed by Sun Microsystems; typically, they are used with Java and Unix, whereas Windows PCs typically use ".wav" files that were developed by Microsoft. Investigate the MATLAB functions **audioread**, **audioinfo**, and **audiowrite**.

Investigate the **colorcube** function, which returns a colormap with regularly spaced R, G, and B colors.

Investigate the **imfinfo** function, which will return information about an image file in a structure variable.

Investigate how colormaps work with image matrices of types **uint8** and **uint16**.

In addition to true color images and indexed images into a colormap, a third type of image is an intensity image, which is used frequently for grayscale images. Investigate how to use the image scale function **imagesc**.

The **uibuttongroup** function is used specifically to group together buttons; other objects can be grouped together similarly using the **uipanel** function. Investigate how this works.

When a GUI has a lot of objects, creating a structure to store the handles can be useful. Investigate the **guihandles** function which accomplishes this.

Investigate the **uitable** function. Use it to create a GUI that demonstrates a matrix operation.

Beginning with MATLAB Version R2012b, GUIs can be packaged as apps! Under the Search Documentation under GUI Building, read how to do this in the category "Packaging GUIs as Apps." Apps can be shared with other users. There is also a lot of information on apps (creating them, downloading them, modifying them, and so forth) under "Desktop Environment." ■

SUMMARY

COMMON PITFALLS

- Confusing true color and colormap images.
- Forgetting that **uicontrol** object positions are within the Figure Window, not within the screen.

PROGRAMMING STYLE GUIDELINES

- Make a GUI invisible while it is being created, so that everything becomes visible at once.

MATLAB Functions and Commands		
colormap	uipanel	laughter
parula	uibuttongroup	splat
jet	root	train
image	uicontrol	handel
pink	movegui	sound
imread	chirp	
imwrite	gong	

Exercises

1. Create a custom colormap for a sphere that consists of the first 25 colors in the colormap **jet**. Display **sphere(25)** with a colorbar as seen in Fig. 13.38.
2. Write a script that will create the image seen in Fig. 13.39 using a colormap.
3. Write a function *numimage* that will receive two input arguments: a colormap matrix, and an integer "*n*"; the function will create an image that shows *n*

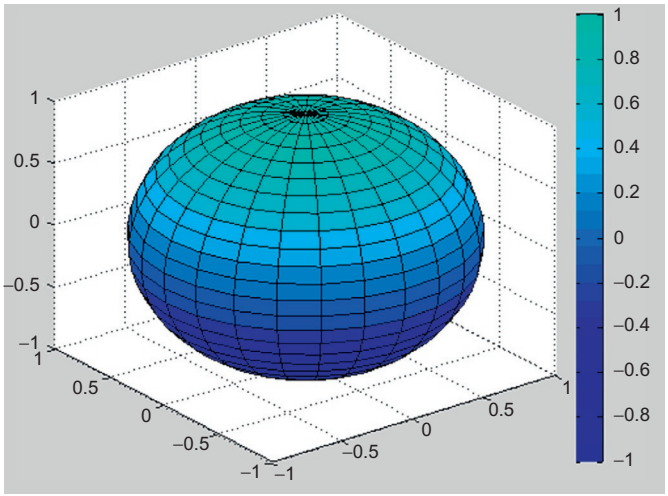


FIGURE 13.38
Custom blue map for a sphere.

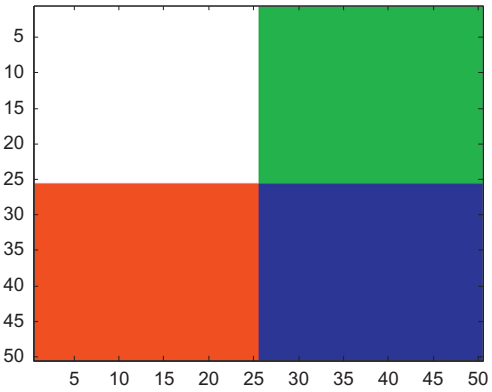


FIGURE 13.39
Image displaying four colors using a custom colormap.

"rings" of color, using the first *n* colors from the colormap. For example, if the function is called as follows:

```
>> cm = [0 0 0; 1 0 0; 0 1 0; 0 0 1; ...
1 1 0; 1 0 1; 0 1 1];
>> numimage(cm,5)
```

the image as seen in Fig. 13.40 will be created. Each "ring" has the thickness of one pixel. In this case, since *n* was 5, the image shows the first 5 colors from the colormap: the outermost ring is the first color, the next ring is the second color, and the innermost pixel is the fifth color. Note that since *n* was 5, the image matrix is 9×9.

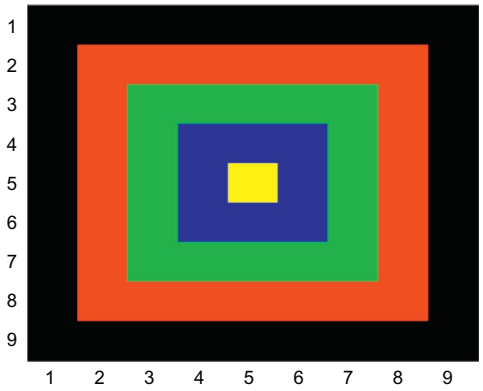


FIGURE 13.40
Color rings.

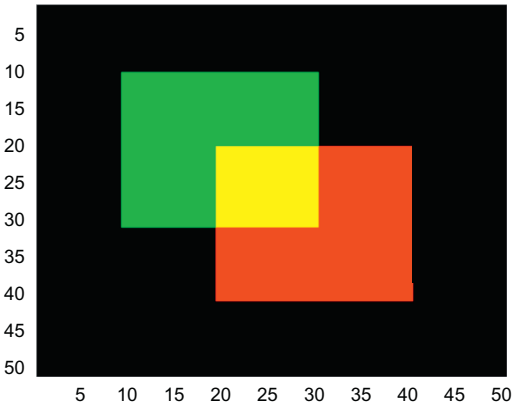


FIGURE 13.41
Green, yellow, and red blocks.

4. Write a script that would produce the 50×50 "image" seen in Fig. 13.41, using the RGB or true color method (NOT the colormap method).
5. A script *rancolors* displays random colors in the Figure Window as shown in Fig. 13.42. It starts with a variable *nColors* which is the number of random colors to display (e.g., below this is 10). It then creates a colormap variable *mycolormap*, which has that many random colors, meaning that all three of the color components (red, green, and blue) are random real numbers in the range from 0 to 1. The script then displays these colors in an image in the Figure Window.

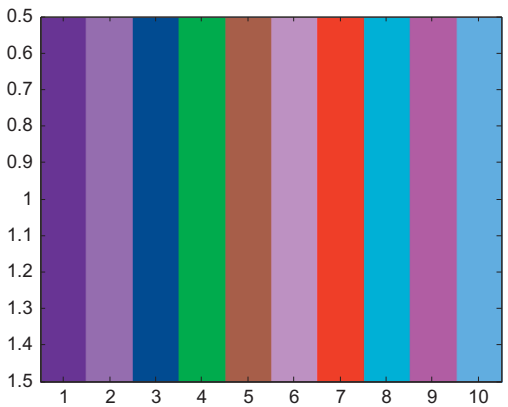


FIGURE 13.42
Rainbow of random colors.

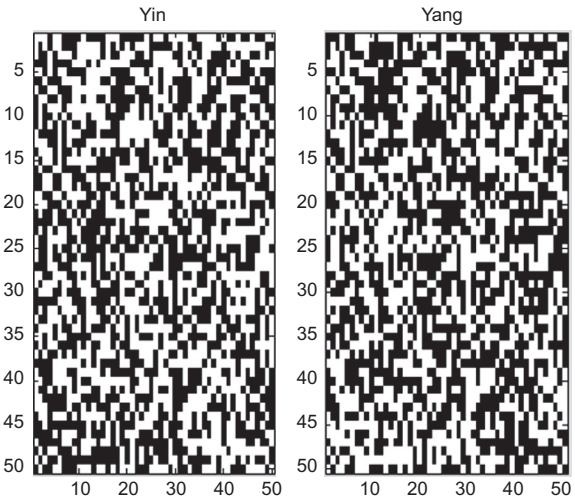


FIGURE 13.43
Reverse black and white pixels.

6. Write a script that will create a colormap that has just two colors: white and black. The script will then create a 50×50 image matrix in which each element is randomly either white or black. In one Figure Window, display this image on the left. On the right, display another image matrix in which the colors have been reversed (all white pixels become black and vice versa). For example, the images might look like Fig. 13.43 (the axes are defaults; note the titles): Do not use any loops or **if** statements. For the image matrix that you created, what would you expect the overall mean of the matrix elements to be?
7. Write a script that will show shades of green and blue as seen in Fig. 13.44. First, create a colormap that has 30 colors (10 blue, 10 aqua, and then 10 green).

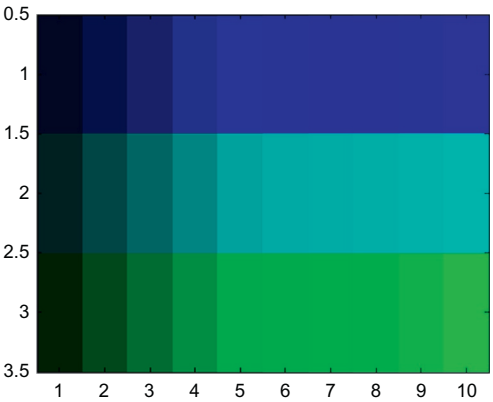


FIGURE 13.44
Shades of blue, aqua, and green.

There is no red in any of the colors. The first 10 rows of the colormap have no green, and the blue component iterates from 0.1 to 1 in steps of 0.1. In the second 10 rows, both the green and blue components iterate from 0.1 to 1 in steps of 0.1. In the last 10 rows, there is no blue, but the green component iterates from 0.1 to 1 in steps of 0.1. Then, display all of the colors from this colormap in a 3×10 image matrix in which the blues are in the first row, aquas in the second, and greens in the third, as follows (the axes are the defaults). Do not use loops.

8. A part of an image is represented by an $n \times n$ matrix. After performing data compression and then data reconstruction techniques, the resulting matrix has values that are close to but not exactly equal to the original matrix. For example, the following 4×4 matrix variable *orig_im* represents a small part of a true color image, and *fin_im* represents the matrix after it has undergone data compression and then reconstruction.

```
orig_im =
   156    44   129    87
    18   158   118   102
    80    62   138    78
   155   150   241   105

fin_im =
   153    43   130    92
    16   152   118   102
    73    66   143    75
   152   155   247   114
```

Write a script that will simulate this by creating a square matrix of random integers, each in the range from 0 to 255. It will then modify this to create the new matrix by randomly adding or subtracting a random number (in a relatively small range, say 0–10) from every element in the original matrix. Then, calculate the average difference between the two matrices.

9. It is sometimes difficult for the human eye to perceive the brightness of an object correctly. For example, in Fig. 13.45, the middle of both images is of the same color, and yet, because of the surrounding colors, the one on the left looks lighter than the one on the right. Write a script to generate a Figure Window similar to this one. Two 3×3 matrices were created. Use **subplot** to display both images side-by-side (the axes shown here are the defaults). Use the RGB method.

10. Put a JPEG file in your Current Folder and use **imread** to load it into a matrix. Calculate and print the **mean** separately of the red, green, and blue components in the matrix.

11. Some image acquisition systems are not very accurate, and the result is ***noisy*** images. To see this effect, put a JPEG file in your Current Folder and use **imread** to load it. Then, create a new image matrix by randomly adding or subtracting a value *n* to every element in this matrix. Experiment with

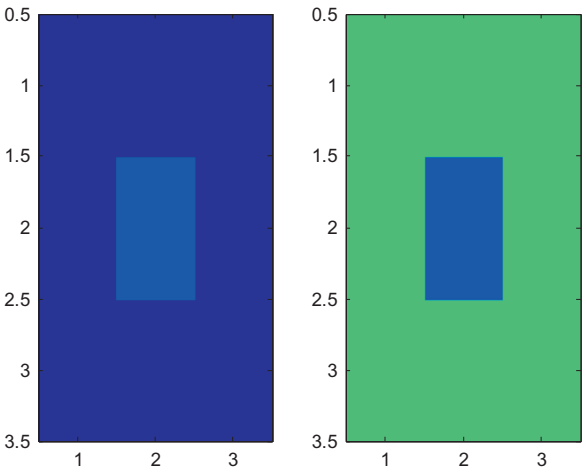


FIGURE 13.45
Depiction of brightness perception.

different values of *n*. Create a script that will use **subplot** to display both images side-by-side.

12. Put a JPEG file into your Current Folder. Type in the following script, using your own JPEG file name.

```
I1 = imread('xxx.jpg');
[r c h] = size(I1);
Inew(:,:,:) = I1(:,c:-1:1,:);
figure(1)
subplot(2,1,1)
image(I1);
subplot(2,1,2)
image(Inew);
```

Determine what the script does. Put comments into the script to explain it step-by-step.

13. Write a function that will create a simple GUI with one static text box near the middle of the Figure Window. Put your name in the string and make the background color of the text box white.
14. Write a function that will create a GUI with one editable text box near the middle of the Figure Window. Put your name in the string. The GUI should have a callback function that prints the user's string twice, one under the other.
15. Fill in the callback function so that it gets the value of the slider, prints that value in the text box, and uses it to set the LineWidth of the plot (so, e.g., if the slider value is its maximum, the line width of the plot would be 5).

```
function sliderlinewidth
f =  figure('Visible', 'off','Position',[20,20,500,400]);
```

```
slhan = uicontrol('Style','slider','Units','Normalized',...
    'Position',[.3 .3 .4 .1], ...
    'Min', 1, 'Max', 5,'Value',3,'Callback', @callbackfn);
slval = uicontrol('Style','text',...
    'Units','Normalized','Position', [.4 .1 .2 .1]);
axhan = axes('Units', 'Normalized','Position', [.3 .5 .4 .3]);
x = -2*pi:0.1:2*pi;
y = cos(x);
phan = plot(x,y)

set(f,'Visible','on');

function callbackfn(source,eventdata)

 end
end
```

16. Write a function that creates a GUI to calculate the area of a rectangle. It should have edit text boxes for the length and width, and a push button that causes the area to be calculated and printed in a static text box.
17. Write a function that creates a simple calculator with a GUI. The GUI should have two editable text boxes in which the user enters numbers. There should be four pushbuttons to show the four operations (+, −, *, /). When one of the four pushbuttons is pressed, the type of operation should be shown in a static text box between the two editable text boxes and the result of the operation should be displayed in a static text box. If the user tries to divide by zero, display an error message in a static text box.
18. Modify any example GUI to use the 'Horizontal Alignment' property to left-justify text within an edit text box.
19. The wind chill factor (WCF) measures how cold it feels with a given air temperature T (in °F) and wind speed (V, in miles/h). The formula is approximately

    $$\text{WCF} = 35.7 + 0.6\,T - 35.7\,\left(V^{0.16}\right) + 0.43\,T\,\left(V^{0.16}\right)$$

    Write a GUI function that will display sliders for the temperature and wind speed. The GUI will calculate the WCF for the given values, and display the result in a text box. Choose appropriate minimum and maximum values for the two sliders.
20. Write a GUI function that will graphically demonstrate the difference between a **for** loop and a **while** loop. The function will have two push buttons: one that says 'for,' and the other that says 'while.' There are two separate callback functions, one associated with each of the pushbuttons. The callback function associated with the 'for' button prints the integers 1 through 5, using **pause(1)** to pause for 1 s between each, and then prints 'Done.' The callback function associated with the 'while' button prints integers beginning with 1 and also pauses between each. This function, however, also has another pushbutton that says 'mystery' on it. This function continues printing integers until the 'mystery' button is pushed, and then it prints 'Finally!'

21. Write a function that will create a GUI in which there is a plot of cos(*x*). There should be two editable text boxes in which the user can enter the range for *x*.
22. Write a function that will create a GUI in which there is a plot. Use a button group to allow the user to choose among several functions to plot.
23. Write a GUI function that will create a **rectangle** object. The GUI has a slider on top that ranges from 2 to 10. The value of the slider determines the width of the **rectangle**. You will need to create axes for the rectangle. In the callback function, use **cla** to clear the children from the current axes so that a thinner rectangle can be viewed.
24. Write a GUI that displays an image in which all of the elements are of the same color. Put 3 sliders in that allow the user to specify the amount of red, green, and blue in the image. Use the RGB method.
25. Put two different JPEG files into your Current Folder. Read both into matrix variables. To superimpose the images, if the matrices are of the same size, the elements can simply be added element-by-element. However, if they are not of the same size, one method of handling this is to crop the larger matrix to be of the same size as that of the smaller, and then add them. Write a script to do this.

    In a random walk, every time a "step" is taken, a direction is randomly chosen. Watching a random walk as it evolves, by viewing it as an image, can be very entertaining. However, there are actually very practical applications of random walks; they can be used to simulate diverse events such as the spread of a forest fire or the growth of a dendritic crystal.
26. The following function simulates a "random walk," using a matrix to store the random walk as it progresses. To begin with all elements are initialized to 1. Then, the "middle" element is chosen to be the starting point for the random walk; a 2 is placed in that element. (Note: these numbers will eventually represent colors.) Then, from this starting point another element next to the current one is chosen randomly and the *color* stored in that element is incremented; this repeats until one of the edges of the matrix is reached. Every time an element is chosen for the next element, it is done randomly by either adding or subtracting one to/from each coordinate (*x* and *y*), or leaving it alone. The resulting matrix that is returned is an *n* by *n* matrix.

ranwalk.m

```
function walkmat = ranwalk(n)
walkmat = ones(n);
x = floor(n/2);
y = floor(n/2);
color = 2;
walkmat(x,y) = color;
while x ~= 1 && x ~= n && y ~= 1 && y ~= n
    x = x + randi([-1 1]);
    y = y + randi([-1 1]);
    color = color + 1;
    walkmat(x,y) = mod(color,65);
end
```

You are to write a script that will call this function twice (once passing 8 and once passing 100) and display the resulting matrices as images side-by-side. Your script must create a custom colormap that has 65 colors; the first is white and the rest are from the colormap **jet**. For example, the result may look like Fig. 13.46. (Note that with the 8×8 matrix the colors are not likely to get out of the blue range, but with 100×00 it cycles through all colors multiple times until an edge is reached.):

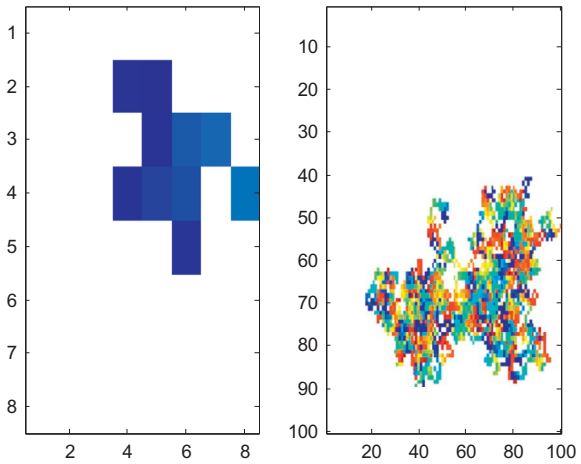


FIGURE 13.46
Random walk.

27. Use App Designer to create a text editor. Create an app that has a large text box as seen in Fig. 13.47. Under it there will be a slider that controls the font size of the text, and buttons to make the text bold and/or italic. Start by dragging a text box into the design area. Click on the box, and then in Design View look at the Edit Field (Text) Properties browser. By changing the properties such as the Style, Name, and Size, and then inspecting the code in Code View, you can see what to change in the callback functions.
28. Create a stoplight app as seen in Fig. 13.48. There are two pushbuttons labeled 'Stop' and 'Go,' and three lamps. When the 'Go' button is pushed, the green lamp is lit. When the 'Stop' button is pushed, the yellow lamp is lit briefly, and then the red lamp is lit.
29. Load two of the built-in MAT-file sound files (e.g., **gong** and **chirp**). Store the sound vectors in two separate variables. Determine how to concatenate these so that the **sound** function will play one immediately followed by the other; fill in the blank here:

```
sound(    , 8192)
```

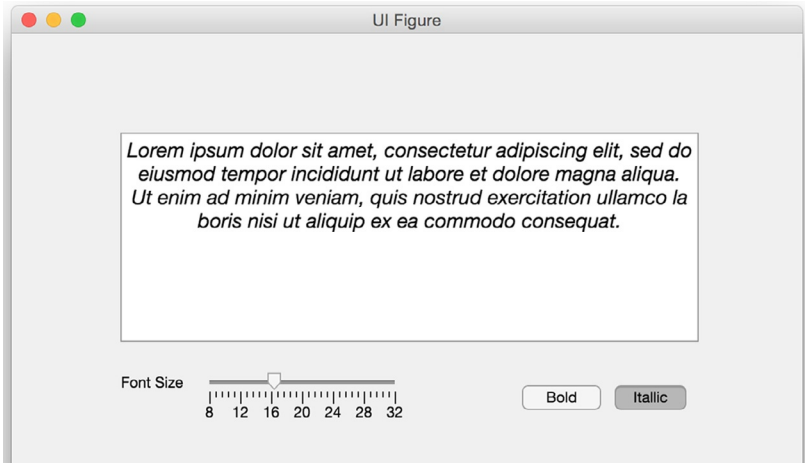


FIGURE 13.47
Text editor app.

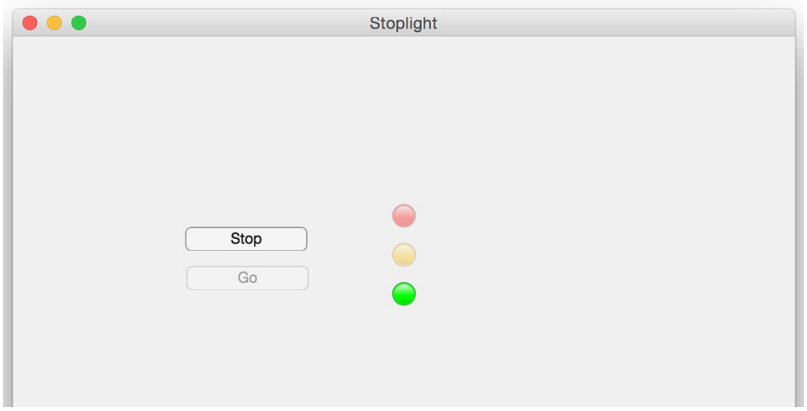


FIGURE 13.48
Stoplight app.

30. The following function *playsound* below plays one of the built-in sounds. The function has a cell array that stores the names. When the function is called, an integer is passed, which is an index into this cell array indicating the sound to be played. The default is 'train,' so if the user passes an invalid index, the default is used. The appropriate MAT-file is loaded. If the user passes a second argument, it is the frequency at which the sound should be played (otherwise, the default frequency is used). The function prints what sound is

about to be played and at which frequency, and then actually plays this sound. You are to fill in the rest of the following function. Here are examples of calling it (you can't hear it here, but the sound will be played!)

```
>> playsound(-4)
You are about to hear train at frequency 8192.0
>> playsound(2)
You are about to hear gong at frequency 8192.0
>> playsound(3,8000)
You are about to hear laughter at frequency 8000.0
```

playsound.m

```
function playsound(caind, varargin)
% This function plays a sound from a cell array
% of mat-file names
% Format playsound(index into cell array) or
%     playsound(index into cell array, frequency)
% Does not return any values

soundarray = {'chirp','gong','laughter','splat','train'};
if caind < 1 || caind > length(soundarray)
    caind = length(soundarray);
end
mysound = soundarray{caind};
eval(['load ' mysound])

% Fill in the rest
```